

Unidade IV

7 DEPURAÇÃO E TESTE DE ALGORITMOS

A depuração e o teste de algoritmos são etapas cruciais no desenvolvimento de software em Python, permitindo que programadores **identifiquem e resolvam problemas**, além de garantir que o código funcione conforme o esperado. Essas práticas são especialmente importantes na implementação de algoritmos, na qual erros lógicos ou comportamentos inesperados podem levar a resultados incorretos ou falhas no programa. Python oferece uma série de ferramentas e abordagens que tornam o processo de depuração e teste mais eficiente e acessível.

Depuração é o processo de localizar e corrigir erros, conhecidos como **bugs**, no código. Esses erros podem surgir por diversas razões, como falhas na lógica, valores inesperados em variáveis ou o uso incorreto de estruturas de dados. Em Python, uma das maneiras mais simples de depurar é usar a função `print` para inspecionar valores de variáveis ou o fluxo de execução em diferentes pontos do código. Embora rudimentar, essa abordagem é muito adotada, pois permite verificar rapidamente o estado do programa sem o uso de ferramentas adicionais. No entanto, para projetos maiores ou mais complexos, o uso de depuradores (debuggers) é mais adequado.

O depurador embutido de Python - `pdb` - é uma ferramenta poderosa que permite ao desenvolvedor pausar a execução do programa, inspecionar variáveis em tempo real e executar o código passo a passo. Com comandos como `break` para definir pontos de parada e `step` para avançar instrução por instrução, o `pdb` oferece controle detalhado sobre a execução do código, facilitando a identificação de problemas em algoritmos. IDEs como PyCharm, VS Code e outros também têm depuradores visuais que integram funcionalidades como inspeção de variáveis e controle de fluxo, tornando a experiência mais intuitiva.

O teste de algoritmos é outra parte essencial do desenvolvimento, garantindo que o código produza os resultados esperados para diferentes entradas. Python tem um ecossistema rico para testes, sendo a biblioteca `unittest` uma das mais utilizadas. Com `unittest`, é possível escrever casos de teste que verificam se as funções e os métodos do programa se comportam corretamente. Além disso, frameworks como `pytest` e `doctest` oferecem funcionalidades adicionais e maior flexibilidade, permitindo escrever e executar testes de forma eficiente.

Testar algoritmos em Python geralmente envolve o fornecimento de entradas específicas e a verificação dos resultados obtidos em comparação com os resultados esperados. Em algoritmos que manipulam dados complexos ou têm diversas ramificações lógicas, um pequeno erro pode se propagar e comprometer a integridade do programa. Além de verificar a correção, os testes ajudam a validar o desempenho e a eficiência do algoritmo, sobretudo em casos de grande volume de dados.

7.1 Técnicas de depuração em Python

A **depuração** é um processo essencial no desenvolvimento de software, e em Python existem diversas técnicas que ajudam os programadores a identificar e corrigir problemas em seu código de forma eficiente. Essas técnicas variam desde métodos básicos, como o uso de impressões no console, até ferramentas avançadas que permitem inspecionar e manipular o estado de execução do programa em tempo real. Cada técnica tem seu lugar dependendo da complexidade do problema e do ambiente de desenvolvimento, mas todas têm como objetivo tornar o código mais robusto e funcional.



A expressão **bug** para se referir a erros em software tem uma origem curiosa e histórica, remontando à era inicial da computação. Embora o termo **bug** já fosse utilizado anteriormente em engenharia e mecânica para descrever falhas ou problemas inesperados em dispositivos, ele ganhou destaque específico no contexto da computação por conta de um incidente famoso envolvendo um computador.

Em 9 de setembro de 1947, os engenheiros que trabalhavam no Harvard Mark II, um dos primeiros computadores eletromecânicos, encontraram um problema no funcionamento da máquina. Após uma inspeção minuciosa, descobriram que uma mariposa havia ficado presa entre os relés do computador, causando falhas elétricas. Eles removeram o inseto e registraram o ocorrido no diário de bordo, descrevendo-o como o primeiro caso real de bug encontrado. A mariposa foi anexada ao diário e tornou-se parte da história.

Desde então, o termo **bug** passou a ser amplamente utilizado para descrever falhas ou erros no comportamento de software e hardware. A resolução de tais problemas, conhecida como **debugging**, também deriva diretamente dessa história. Hoje o termo é um elemento universal no vocabulário de programadores e engenheiros, e embora a origem esteja enraizada em uma falha literal causada por um inseto, o uso moderno do termo é puramente figurativo. A expressão reflete a ideia de algo pequeno, inesperado e difícil de detectar que compromete o funcionamento adequado de um sistema.

Uma das abordagens mais simples e comuns para depuração é o uso de mensagens no console com a função `print`. Ao adicionar `print` em pontos estratégicos do código, é possível inspecionar o valor de variáveis e verificar o fluxo de execução. Essa técnica é útil para identificar erros simples e para confirmar se determinada parte do programa está sendo executada como esperado.

Ao lidar com um sistema de viagem no tempo, o desenvolvedor frequentemente se depara com comportamentos inesperados: uma variável que não assume o valor esperado, um trecho de código que parece jamais ser executado ou até mesmo um cálculo que retorna resultados estranhos. Nessas situações, o uso de `print` para depuração pode ser uma aliada valiosa.

O exemplo a seguir mostra um código de simulação simples, no qual o viajante do tempo tenta ajustar seu destino. O programa solicita um ano de destino, realiza alguns cálculos relacionados à energia necessária e avalia o modo de viagem. Caso algo não saia conforme o esperado, inserir `print` em pontos críticos do código ajudará a entender em que momento as coisas desandam.

```
from datetime import datetime

# Obtém o ano atual, considerando-o como ponto de referência do presente.
ano_atual = datetime.now().year
print(f"[DEBUG] Ano atual obtido: {ano_atual}") # Depuração: Confirma o ano atual
extraído do sistema.

# Solicita ao usuário o ano de destino e converte para inteiro.
entrada_ano = input("Insira o ano de destino: ")
try:
    ano_destino = int(entrada_ano)
    print(f"[DEBUG] Ano destino inserido pelo usuário: {ano_destino}") #
    Depuração: Checa se a conversão para int ocorreu corretamente.
except ValueError:
    # Caso o usuário insira algo que não seja um número inteiro, informa o erro.
    print("[DEBUG] Erro na conversão do ano de destino, entrada inválida:",
    entrada_ano)
    print("Ano inválido. Encerrando o programa.")
    exit()

# Calcula a diferença entre o ano atual e o ano de destino. Esse cálculo pode
# ser fundamental para determinar energia, risco, etc.
diferenca = ano_destino - ano_atual
print(f"[DEBUG] Diferença temporal calculada: {diferenca}") # Depuração: Verifica
se a diferença está correta.

# Suponhamos que exista um fator energético simples, como 10 unidades de energia
# por ano de diferença.
energia_necessaria = abs(diferenca) * 10
print(f"[DEBUG] Energia necessária calculada: {energia_necessaria}") #
Depuração: Confirma o valor calculado.

# Solicita o modo de viagem: "instantaneo" ou "gradual". Caso o usuário insira
# algo diferente, lidamos separadamente.
modo = input("Escolha o modo de viagem (instantaneo/gradual): ").strip().lower()
print(f"[DEBUG] Modo de viagem inserido: {modo}") # Depuração: Garante que o
valor lido é o esperado.

if modo == "instantaneo":
    # Caso o modo seja instantâneo, não há necessidade de esperar ou preparar
    # etapas intermediárias.
    print("[DEBUG] O modo de viagem selecionado é instantâneo, pulando etapas
    intermediárias.")
```

```
elif modo == "gradual":
    # Caso o modo seja gradual, pode-se supor que o programa realizaria cálculos
    adicionais, pausas, ou validações.
    print("[DEBUG] O modo de viagem selecionado é gradual, preparando a
    sequência de saltos intermediários.")
else:
    # Caso o usuário insira um modo não reconhecido, informa o erro.
    print(f"[DEBUG] Modo desconhecido: {modo}, encerrando.")
    print("Modo de viagem inválido. Encerrando o programa.")
    exit()

# Aqui, supomos que tudo está correto, e o programa pode seguir para o disparo
da viagem.
# Antes de concluir, uma checagem final dos valores que serão utilizados.
print("[DEBUG] Preparando para iniciar a viagem.")
print(f"[DEBUG] Ano atual: {ano_atual}, Ano destino: {ano_destino}, Energia:
{energia_necessaria}, Modo: {modo}")

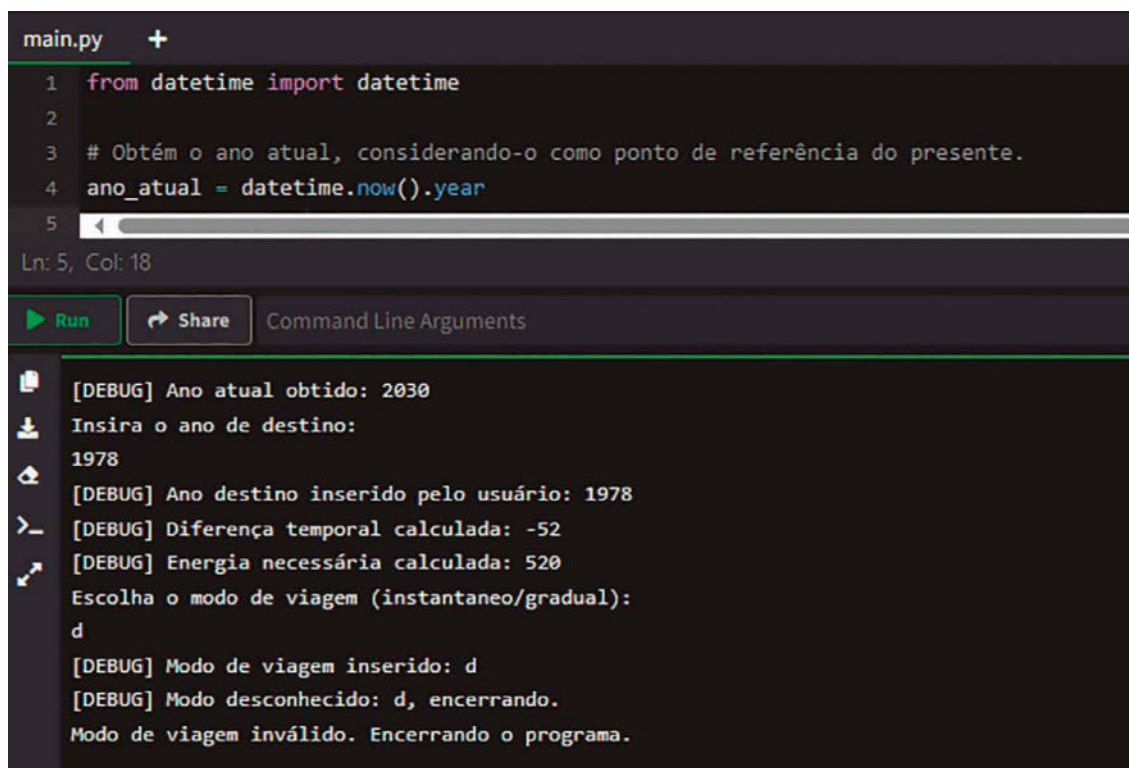
# Se tudo estiver certo, o programa indicaria o início da viagem. Caso o
desenvolvedor note algo estranho,
# as mensagens [DEBUG] já impressas no console auxiliam na identificação do
problema.
print("Viagem iniciada. Boa sorte na sua aventura temporal!")
```

Figura 132

Nesse código, o uso de `print` para depuração acontece em diversos pontos críticos. No início, quando o ano atual é obtido, a mensagem `[DEBUG]` assegura que essa informação foi corretamente adquirida. Ao receber o ano de destino do usuário, outro `print` confirma a conversão da entrada e, caso haja um erro, o próprio tratamento de exceção exibe uma mensagem de depuração. Quando o programa calcula a diferença entre os anos, outra linha de `print` exibe o resultado, permitindo verificar se a lógica de subtração funciona como o esperado. O mesmo se aplica ao cálculo da energia necessária e à verificação do modo de viagem escolhido.

Se em algum momento o programa se comportar estranhamente, por exemplo, calculando uma energia que pareça absurda ou não reconhecendo o modo de viagem, essas mensagens de depuração ajudarão a entender por que isso ocorre. O desenvolvedor pode, a partir dessas saídas, inferir se o erro vem de uma entrada não tratada, de uma conversão incorreta ou de uma lógica falha em alguma parte do código.

Embora a técnica de adicionar `print` em pontos-chave do código seja simples, ela é extremamente útil no dia a dia da programação, especialmente quando o desenvolvedor ainda não está usando ferramentas mais sofisticadas de depuração. O importante é saber onde inserir essas mensagens, mantendo-as claras, objetivas e, se possível, removendo ou comentando-as uma vez que o problema seja identificado e corrigido, de modo a não poluir a saída final do programa em produção. A figura a seguir apresenta a execução do programa:



The screenshot shows a code editor with a file named `main.py`. The code is as follows:

```
1 from datetime import datetime
2
3 # Obtém o ano atual, considerando-o como ponto de referência do presente.
4 ano_atual = datetime.now().year
5
```

Below the code editor, there is a console window showing the output of the program. The output is as follows:

```
[DEBUG] Ano atual obtido: 2030
Insira o ano de destino:
1978
[DEBUG] Ano destino inserido pelo usuário: 1978
[DEBUG] Diferença temporal calculada: -52
[DEBUG] Energia necessária calculada: 520
Escolha o modo de viagem (instantaneo/gradual):
d
[DEBUG] Modo de viagem inserido: d
[DEBUG] Modo desconhecido: d, encerrando.
Modo de viagem inválido. Encerrando o programa.
```

Figura 133 – Utilização de print para depuração de código

Apesar de ser amplamente utilizada, essa abordagem pode rapidamente se tornar confusa em projetos maiores ou mais complexos, especialmente se muitas mensagens forem adicionadas, tornando difícil rastrear o problema.

Para cenários mais avançados, Python oferece o depurador embutido `pdb` (Python Debugger). O `pdb` permite que o desenvolvedor execute o programa passo a passo, inspecione variáveis em tempo real e defina pontos de parada (breakpoints) onde a execução será interrompida. Tal recurso é especialmente útil quando o erro não é óbvio ou ocorre em condições específicas que são difíceis de reproduzir. Com comandos como `list` para visualizar o código ao redor do ponto atual, `step` para avançar instrução por instrução e `continue` para retomar a execução até o próximo breakpoint, o `pdb` oferece um controle detalhado sobre a execução. IDEs modernas, como PyCharm, Visual Studio Code e Jupyter Notebooks, integram depuradores gráficos que tornam esse processo mais intuitivo, permitindo que o desenvolvedor inspecione variáveis e observe o fluxo de execução visualmente.



Saiba mais

Para o depurador embutido do Python, o `pdb`, a documentação oficial oferece um guia completo sobre os comandos e recursos disponíveis. É possível aprender a definir breakpoints, avançar passo a passo no código e inspecionar variáveis em tempo real:

Disponível em: <https://shre.ink/gapd>. Acesso em: 12 dez. 2024.

No PyCharm, uma das IDEs mais populares para Python, o depurador integrado é altamente visual e oferece funcionalidades como inspeção de variáveis, pontos de interrupção condicionais e visualização de pilhas de chamadas. O guia oficial da JetBrains fornece instruções detalhadas para usar o depurador em diferentes cenários:

Disponível em: <https://shre.ink/gafJ>. Acesso em: 12 dez. 2024.

Para o Visual Studio Code (VS Code), que também suporta Python de forma robusta, o depurador oferece integração com breakpoints, execução passo a passo e suporte a variáveis interativas. O site oficial do VS Code inclui tutoriais e exemplos para ajudar os desenvolvedores a configurar e usar o depurador:

Disponível em: <https://shre.ink/gafv>. Acesso em: 12 dez. 2024.

A seguir apresentaremos o mesmo código do exemplo anterior, porém utilizando o Python Debugger. Ao invés de `print` marcados com `[DEBUG]`, usamos `breakpoint()` (disponível no Python 3.7+) para interromper a execução naquele ponto. Caso você esteja em uma versão anterior à 3.7, poderia usar `import pdb; pdb.set_trace()` no lugar de `breakpoint()`.

Uma vez interrompida a execução, o desenvolvedor pode inspecionar variáveis digitando seus nomes no prompt do depurador, avançar com `n` (next), entrar em funções com `s` (step), continuar a execução com `c` (continue), entre outros comandos. Assim, toda a análise que antes era limitada a `print` agora é muito mais interativa e flexível.

```
from datetime import datetime

ano_atual = datetime.now().year
# Ao invés de printar o ano atual, utilizamos breakpoint para inspecionar seu
valor interativamente no depurador.
breakpoint() # Ao chegar aqui, o programa interrompe. Podemos digitar 'ano_
atual' no prompt e verificar seu valor.

entrada_ano = input("Insira o ano de destino: ")
try:
    ano_destino = int(entrada_ano)
except ValueError:
    print("Ano inválido. Encerrando o programa.")
    exit()

# Podemos inserir outro breakpoint aqui para inspecionar ano_destino e garantir
que foi convertido corretamente.
breakpoint()

diferenca = ano_destino - ano_atual
energia_necessaria = abs(diferenca) * 10

# Coloca-se outro breakpoint após o cálculo para verificar se a lógica da
diferença e do cálculo de energia está correta.
breakpoint()

modo = input("Escolha o modo de viagem (instantaneo/gradual): ").strip().lower()

# Mais um breakpoint antes de checar o modo, permitindo inspecionar a variável
'modo' e se necessário manipular seu valor.
breakpoint()

if modo == "instantaneo":
    print("O modo de viagem selecionado é instantâneo. Preparando salto
    imediato.")
elif modo == "gradual":
    print("O modo de viagem selecionado é gradual. Preparando sequência de
    saltos intermediários.")
else:
    print("Modo de viagem inválido. Encerrando o programa.")
    exit()

# Por fim, antes de iniciar a viagem, outro breakpoint para checar todas as
variáveis e o estado do programa.
breakpoint()

print("Viagem iniciada. Boa sorte na sua aventura temporal!")
```

Figura 134

Nesse código, cada `breakpoint()` age como um ponto de parada. Ao executar o programa em um terminal (por exemplo, `python3 script.py`), ao chegar a um `breakpoint()`, o Python entra no modo interativo do depurador. Nele o desenvolvedor pode:

- Digitar o nome de variáveis (como `ano_atual`, `ano_destino`, `diferenca`, `energia_necessaria`, `modo`) para ver seus valores atuais.
- Utilizar comandos do `pdb` como `n` (next) para avançar linha a linha, `s` (step) para entrar em funções, `c` (continue) para retomar a execução até o próximo breakpoint ou o final do programa, `l` (list) para ver o código-fonte em volta do ponto atual, entre outras possibilidades.
- Alterar o valor de variáveis no depurador (por exemplo, `ano_destino = 2050`) antes de continuar, testando como o programa reagiria a entradas diferentes sem precisar reiniciar a execução desde o início.



Lembrete

Em algoritmos que manipulam dados complexos ou têm diversas ramificações lógicas, um pequeno erro pode se propagar e comprometer a integridade do programa. Além de verificar a correção, os testes ajudam a validar o desempenho e a eficiência do algoritmo, sobretudo em casos de grande volume de dados.

Na figura a seguir podemos visualizar a execução do código com o `pdb` ativo:

```
main.py +
1 from datetime import datetime
2
3 ano_atual = datetime.now().year
4 # Ao invés de printar o ano atual, utilizamos breakpoint para inspecionar seu valor interativamente no depurador.
5 breakpoint() # Ao chegar aqui, o programa interrompe. Podemos digitar 'ano_atual' no prompt e verificar seu valor

Ln: 40, Col: 1

[Stop] [Share] Command Line Arguments

> /home/repl/622e70c9-e9e3-4f99-8eff-e2501978b310/main.py(7)<module>()
-> entrada_ano = input("Insira o ano de destino: ")
(Pdb)
c
> Insira o ano de destino:
1978
> /home/repl/622e70c9-e9e3-4f99-8eff-e2501978b310/main.py(17)<module>()
-> diferenca = ano_destino - ano_atual
(Pdb)
c
> /home/repl/622e70c9-e9e3-4f99-8eff-e2501978b310/main.py(23)<module>()
-> modo = input("Escolha o modo de viagem (instantaneo/gradual): ").strip().lower()
(Pdb)
```

Figura 135 – Uso do Python Debugger na depuração de código

Outra técnica importante de depuração envolve o uso de exceções. Python tem um sistema robusto para tratar e manipular erros por meio de blocos `try-except`. Durante a execução, se uma exceção é levantada, o programador pode capturá-la em um bloco `except` e exibir informações detalhadas

sobre o erro, como a mensagem associada e o local onde ele ocorreu. Essa técnica ajuda a identificar problemas e evita que o programa falhe de forma abrupta, permitindo um manuseio mais gracioso dos erros. Para diagnósticos ainda mais detalhados, a biblioteca `traceback` pode ser utilizada para exibir a pilha de chamadas completa, indicando o caminho exato até o erro.

A depuração também pode ser facilitada com ferramentas externas e bibliotecas especializadas. Por exemplo, o módulo `logging` permite registrar eventos no programa em diferentes níveis de severidade, como `DEBUG`, `INFO`, `WARNING`, `ERROR` e `CRITICAL`. Esses eventos servem para monitorar o comportamento do programa ao longo do tempo, especialmente em sistemas grandes ou distribuídos. Diferentemente do `print`, que exibe informações diretamente no console, o `logging` pode direcionar as mensagens para arquivos ou sistemas de monitoramento, tornando-o ideal para depuração em ambientes de produção.

7.2 Testes automatizados simples

Os testes automatizados são uma prática essencial no desenvolvimento de software moderno, permitindo que os desenvolvedores verifiquem automaticamente se as funcionalidades do programa estão funcionando conforme esperado. Entre os tipos mais comuns de testes automatizados estão os **testes unitários**, que se concentram em verificar pequenas partes do código, como funções ou métodos, de forma isolada. Esses testes são simples de implementar e oferecem benefícios significativos, como a detecção precoce de erros e a garantia de que mudanças no código não introduzam problemas inesperados.

A criação de testes unitários envolve escrever pequenos programas que chamam funções ou métodos específicos com diferentes entradas e verificam se os resultados obtidos correspondem aos esperados. Cada teste unitário deve ser focado em uma única funcionalidade ou comportamento do código, garantindo que qualquer falha seja facilmente identificada e localizada. Por exemplo, em uma função que soma dois números, um teste unitário poderia verificar se a soma de 2 e 3 resulta em 5. Esses testes fornecem uma base sólida para validar os fundamentos do programa.

Python oferece bibliotecas robustas para a criação de testes unitários, sendo a mais comum a biblioteca integrada `unittest`. Essa biblioteca permite criar classes que agrupam múltiplos testes e métodos específicos que realizam as verificações. Cada método dentro de uma classe de teste representa um caso de teste, e os resultados são comparados usando as funções de asserção da biblioteca, como `assertEqual`, `assertTrue` ou `assertFalse`. Quando um teste falha, o `unittest` informa exatamente qual parte do código apresentou o problema, facilitando a depuração.

A manutenção de um sistema de viagem no tempo exige o desenvolvimento cuidadoso de suas funcionalidades, bem como a garantia de que essas funções realmente se comportam como o esperado. Em um contexto tão delicado, em que pequenos erros podem gerar paradoxos e distorções irreparáveis na linha temporal, testes se tornam uma ferramenta fundamental. Antes de efetivamente acionar o painel principal da máquina do tempo, é sensato verificar, por meio de testes básicos, se as funções principais estão trabalhando corretamente. Esses testes podem ser simples, mas já fornecem um grau de confiança no código, assegurando que pequenos trechos funcionam conforme o projetado.

O código a seguir apresenta um pequeno conjunto de funções que simulam operações típicas envolvidas no planejamento de uma jornada temporal. Por exemplo, um cálculo de diferença entre o ano atual e o ano de destino, o ajuste de energia necessária para o salto e a validação de modos de viagem (instantâneo ou gradual). Em seguida, demonstra testes básicos, verificando com `assert` se o retorno dessas funções é o esperado.

```
from datetime import datetime

def calcular_diferenca(ano_atual, ano_destino):
    # Esta função recebe o ano atual e o ano de destino e retorna a diferença
    # entre eles.
    # Por exemplo, se ano_atual=2024 e ano_destino=2050, a função retorna 26.
    # Isso é importante, pois diversas rotinas da máquina do tempo dependem
    # dessa diferença temporal,
    # como o cálculo de energia ou a determinação de quantos saltos
    # intermediários serão necessários.
    return ano_destino - ano_atual

def calcular_energia(anos):
    # Esta função recebe um número inteiro que representa quantos anos
    # serão percorridos.
    # Suponhamos que cada ano de diferença exija 10 unidades de energia.
    # Com isso, se anos=20, a função retorna 200.
    # Essa lógica simples é um ponto de partida para algo mais complexo,
    # mas já garante coerência com o esforço energético da viagem.
    return abs(anos) * 10

def validar_modos_viagem(modos):
    # Esta função recebe uma string representando o modo de viagem, e verifica se
    # é válido.
    # Podem existir dois modos válidos: "instantaneo" e "gradual".
    # Caso a string não seja uma dessas duas, a função retorna False, indicando
    # modo inválido.
    modo = modos.strip().lower()
    if modo == "instantaneo" or modo == "gradual":
        return True
    return False

# Código principal, onde realizaremos testes simples das funções definidas acima.
# Suponha que o desenvolvedor esteja configurando o sistema antes de ativar a
# máquina do tempo.
# Ele deseja ter certeza de que o cálculo de diferença de anos, o cálculo de
# energia e a validação do modo funcionam.

# Obtém o ano atual do sistema e define um ano de destino para teste.
ano_atual = datetime.now().year
ano_destino_teste = ano_atual + 30 # Supondo um salto de 30 anos para o futuro,
# por exemplo.

# Testa a função calcular_diferenca. Espera-se que a diferença seja 30.
diferenca_calculada = calcular_diferenca(ano_atual, ano_destino_teste)
assert diferenca_calculada == 30, f"Esperava-se uma diferença de 30, mas
obteve-se {diferenca_calculada}."
# Se este assert falhar, significará que a função não está retornando o
# valor correto,
# indicando um problema lógico a ser investigado antes de prosseguir.
```

```
# Testa a função calcular_energia com base na diferença calculada.
# Esperamos que para 30 anos, a energia seja 30 * 10 = 300.
energia_calculada = calcular_energia(diferenca_calculada)
assert energia_calculada == 300, f"Esperava-se energia de 300, mas obteve-se
{energia_calculada}."
# Caso falhe, significa que a lógica de cálculo de energia está incorreta,
# talvez multiplicando pelo fator errado ou interpretando mal o valor absoluto.

# Testa a função validar_modos_viagem.
# Primeiro, algo que esperamos ser válido: "instantaneo".
resultado_modos = validar_modos_viagem("instantaneo")
assert resultado_modos == True, "Esperava-se True para o modo 'instantaneo'."
# Este assert verifica se o modo reconhecido como válido realmente retorna True.
# Caso falhe, a função de validação não está corretamente reconhecendo
modos válidos.

# Agora testamos com algo inválido, como "rapido".
resultado_modos = validar_modos_viagem("rapido")
assert resultado_modos == False, "Esperava-se False para o modo 'rapido', que não
é reconhecido."
# Este teste garante que modos desconhecidos não sejam aceitos inadvertidamente,
# o que é importante para impedir que o usuário defina um modo inexistente,
comprometendo a coerência do sistema.

# Se todos os assert acima forem executados sem falhas, significa que as funções
básicas funcionam conforme o esperado.
# Não houve mensagens de erro, indicando que as funções passaram nos testes
rudimentares aqui propostos.
print("Todos os testes básicos de função foram bem-sucedidos.")

# Dessa forma, antes de ligar o reator temporal e iniciar uma nova jornada, o
desenvolvedor pode respirar aliviado,
# sabendo que, ao menos nesse nível elementar, a lógica fundamental do programa
está coerente.
# Embora sejam testes extremamente simples, eles servem como ponto de partida
para criar um conjunto mais amplo e completo,
# abrangendo casos extremos, valores de borda e cenários inesperados. Em um
contexto tão delicado quanto a manipulação
# do tempo, garantir a confiabilidade do código é um passo essencial para evitar
catástrofes cronológicas.
```

Figura 136

Nesse exemplo, o cenário se concentra na lógica interna do programa antes mesmo de a máquina do tempo entrar em ação. Foram definidas três simples funções: uma para calcular a diferença entre anos, outra para calcular a energia necessária e outra para validar o modo de viagem. Pequenos testes com `assert` foram escritos para verificar se essas funções produzem o resultado esperado. Se algum teste falha, o `assert` gera um erro, indicando ao desenvolvedor que revise aquela parte da lógica. Caso tudo corra bem, o `print` final informa que todos os testes básicos foram bem-sucedidos.

Assim, mesmo sem frameworks de teste mais elaborados, usar `assert` já oferece um nível mínimo de confiança. A prática incentiva uma mentalidade de testes em estágios iniciais do desenvolvimento, reduzindo a probabilidade de problemas sérios quando o sistema for realmente utilizado para navegar entre séculos e milênios. A figura a seguir ilustra o resultado da execução do código:

```
main.py +
1 from datetime import datetime
2
3 def calcular_diferenca(ano_atual, ano_destino):
4     # Esta função recebe o ano atual e o ano de destino e retorna a diferença entre eles.
5     # Por exemplo, se ano_atual=2024 e ano_destino=2050, a função retorna 26.
6     # Isso é importante, pois diversas rotinas da máquina do tempo dependem dessa diferença temporal,
7     # como o cálculo de energia ou a determinação de quantos saltos intermediários serão necessários.
8     return ano_destino - ano_atual
9
10 def calcular_energia(anos):
11     # Esta função recebe um número inteiro que representa quantos anos serão percorridos.
12     # Suponhamos que cada ano de diferença exija 10 unidades de energia.
13     # Com isso, se anos=20, a função retorna 200.
14     # Essa lógica simples é um ponto de partida para algo mais complexo,
15     # mas já garante coerência com o esforço energético da viagem.
16     return abs(anos) * 10
17
18 def validar_modos_viagem(modos):
19     # Esta função recebe uma string representando o modo de viagem, e verifica se é válido.
20     # Podem existir dois modos válidos: "instantaneo" e "gradual".
21
Ln: 70, Col: 1
```

Run Share Command Line Arguments

Todos os testes básicos de função foram bem-sucedidos.

** Process exited - Return Code: 0 **

Press Enter to exit terminal

Figura 137 – Testes usando assert

Como apresentamos, a simples verificação com `assert` já ajuda, mas conforme o sistema cresce e as funções se tornam mais complexas, pode-se precisar de abordagens mais robustas e padronizadas de teste. É aí que entra o `unittest`, um framework embutido no Python que permite estruturar **casos de teste** em classes, agrupar múltiplos testes e obter relatórios claros sobre o sucesso ou falha de cada trecho de código. Ao usar o `unittest`, é possível criar testes mais organizados, reaproveitáveis e que se integrem a processos automatizados, oferecendo maior confiança no funcionamento do programa.



Observação

Casos de teste, no contexto do desenvolvimento de software, são unidades específicas de verificação projetadas para avaliar se uma funcionalidade, módulo ou método do programa está funcionando corretamente. Em termos simples, um caso de teste é um conjunto de condições, entradas e resultados esperados que descrevem um cenário a ser testado. Ele serve como uma forma sistemática de garantir que o código se comporte conforme o esperado em diferentes situações, incluindo aquelas que podem gerar erros.

Os casos de teste são fundamentais para verificar diferentes cenários de uso de uma funcionalidade. Por exemplo, se você estiver testando uma função que soma dois números, os casos de teste podem incluir somas básicas (como $2 + 3$), somas com números negativos ($-2 + 3$), somas envolvendo zero ($0 + 5$) ou até mesmo entradas inválidas, como strings ou valores nulos. Cada um desses cenários seria um caso de teste distinto, pois representa uma condição que o programa pode encontrar em execução.

Um dos objetivos dos casos de teste é cobrir o máximo possível de situações que o código pode enfrentar no mundo real. Além de verificar funcionalidades específicas, casos de teste ajudam a garantir que mudanças futuras no código não introduzam problemas inesperados. Isso é conhecido como **teste de regressão**. Sempre que uma alteração é feita, os casos de teste existentes podem ser executados novamente para garantir que todas as funcionalidades previamente verificadas continuam funcionando como esperado.

A seguir apresentaremos um código que implementa testes das mesmas funções básicas vistas anteriormente, mas agora utilizando `unittest`. Temos funções que calculam diferenças entre anos, calculam energia para o salto temporal e validam o modo de viagem. Em seguida, definimos uma classe de teste que herda `unittest.TestCase` e cria métodos de teste individuais para cada função. Ao rodar esses testes, o framework `unittest` gerará um relatório, informando quantos testes passaram, quantos falharam e exibindo mensagens de erro detalhadas em caso de problemas. Assim, o desenvolvedor pode, com mais clareza, manter e evoluir o código, confiando de que as funcionalidades críticas são asseguradas.

```
import unittest
from datetime import datetime

def calcular_diferenca(ano_atual, ano_destino):
    # Retorna a diferença entre ano_destino e ano_atual.
    # Exemplo: se ano_atual=2024 e ano_destino=2050, retorna 26.
    # Essa função é essencial para diversos cálculos relacionados ao tempo, pois
    # a partir da diferença é possível estimar complexidade.
    return ano_destino - ano_atual

def calcular_energia(anos):
    # Recebe um número de anos e retorna a energia necessária, presumindo 10
    # unidades por ano.
    # Exemplo: se anos=20, a função retorna 200.
    # Esta lógica simples é um passo inicial para algo mais elaborado no futuro.
    return abs(anos) * 10

def validar_modos_viagem(modos):
    # Verifica se o modo de viagem informado é válido.
    # Pode ser "instantaneo" ou "gradual". Caso contrário, retorna False.
    # Essa verificação impede que o usuário selecione modos inexistentes,
    # preservando a coerência do sistema.
    modo = modos.strip().lower()
    if modo == "instantaneo" or modo == "gradual":
        return True
    return False
```

```
class TesteFuncoesTempo(unittest.TestCase):
    # Esta classe agrupa diversos testes relacionados às funções de manipulação
    de tempo.
    # Ao herdar de unittest.TestCase, cada método test_* será considerado um
    teste individual.

    def setUp(self):
        # O método setUp é executado antes de cada teste.
        # Aqui, obtemos o ano atual e definimos um destino padrão.
        # Isso garante um ambiente inicial coerente para todos os testes,
        reduzindo repetição de código.
        self.ano_atual = datetime.now().year
        self.ano_destino_teste = self.ano_atual + 30

    def test_calcular_diferenca(self):
        # Testa a função calcular_diferenca. Espera-se 30, já que ano_destino_
        teste está 30 anos à frente.
        diferenca = calcular_diferenca(self.ano_atual, self.ano_destino_teste)
        self.assertEqual(diferenca, 30, f"Esperava-se uma diferença de 30, mas
        obteve-se {diferenca}.")

    def test_calcular_energia(self):
        # Testa o cálculo de energia. Se a diferença é 30, energia deve ser
        30*10=300.
        diferenca = 30
        energia = calcular_energia(diferenca)
        self.assertEqual(energia, 300, f"Esperava-se energia de 300, mas obteve-se
        {energia}.")

    def test_validar_modo_viagem_valido(self):
        # Testa se um modo válido ('instantaneo') retorna True.
        resultado = validar_modo_viagem("instantaneo")
        self.assertTrue(resultado, "Esperava-se True para o modo 'instantaneo'.")

    def test_validar_modo_viagem_invalido(self):
        # Testa se um modo inválido ('rapido') retorna False.
        resultado = validar_modo_viagem("rapido")
        self.assertFalse(resultado, "Esperava-se False para o modo 'rapido', não
        reconhecido pelo sistema.")

if __name__ == "__main__":
    # Este bloco permite executar o conjunto de testes chamando python3 nome_do_
    arquivo.py.
    # O unittest rodará todos os métodos test_* da classe TesteFuncoesTempo,
    gerando um relatório sucinto.
    unittest.main()
```

Figura 138

O código apresentado combina a definição de funções úteis para manipulação de tempo e energia com a implementação de testes automatizados para validar o comportamento dessas funções. O código começa definindo três funções:

- `calcular_diferenca`: essa função calcula a diferença entre o ano atual e um ano de destino, retornando o número de anos de diferença. Ela é útil em cenários relacionados ao cálculo de tempo, como determinar o número de anos a serem percorridos em uma viagem temporal. A função assume que a entrada está correta e opera diretamente com números inteiros.
- `calcular_energia`: baseando-se na diferença de anos, essa função calcula a energia necessária para a viagem temporal, presumindo um custo fixo de 10 unidades por ano. O uso da função `abs` garante que a energia seja sempre positiva, independentemente de a viagem ser para o futuro ou para o passado.
- `validar_modos_viagem`: verifica se o modo de viagem fornecido é válido, aceitando apenas os valores `instantaneo` e `gradual`. A função normaliza a entrada, removendo espaços extras e convertendo para letras minúsculas antes da comparação. Essa validação protege o sistema contra entradas incorretas e mantém a consistência.

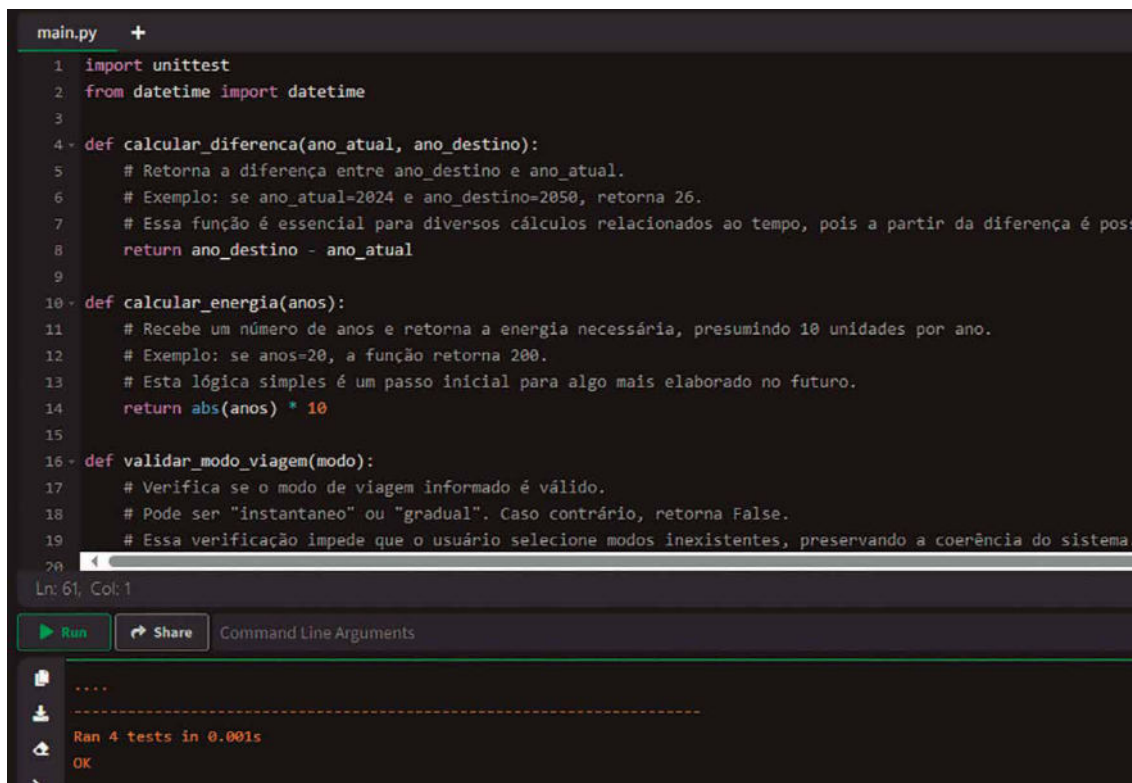
Após definir essas funções, a classe `TesteFuncoesTempo` é implementada para agrupar os testes unitários que verificam o comportamento esperado de cada função. Essa classe herda `unittest.TestCase`, o que permite que os métodos cujo nome começa com `test_` sejam reconhecidos como testes.

A configuração inicial para os testes é feita no método `setUp`, que é executado antes de cada teste. Ele define um ponto de partida consistente, armazenando o ano atual e um destino padrão (30 anos à frente do atual). Isso evita a repetição de código nos métodos de teste individuais. Os métodos de teste incluem:

- `test_calcular_diferenca`: verifica se a função `calcular_diferenca` retorna corretamente 30 anos de diferença, dado o destino definido no `setUp`. O uso de `assertEqual` garante que a saída da função corresponda ao valor esperado.
- `test_calcular_energia`: testa a função `calcular_energia` com uma diferença de 30 anos, esperando um resultado de 300 unidades de energia. Novamente, o `assertEqual` é usado para validar a saída.
- `test_validar_modos_viagem_valido`: confirma que a função `validar_modos_viagem` retorna `True` quando o modo `instantaneo` é fornecido. O método `assertTrue` valida que o resultado é verdadeiro.
- `test_validar_modos_viagem_invalido`: testa um caso no qual a entrada é inválida (`rapido`), esperando que a função retorne `False`. O método `assertFalse` valida o comportamento esperado.

Por fim, o bloco `if __name__ == "__main__":` permite que o script seja executado diretamente. Ele chama `unittest.main()`, que automaticamente descobre e executa todos os métodos de teste definidos na classe, gerando um relatório detalhado sobre o sucesso ou falha de cada

teste. A parte inferior da figura a seguir mostra o resultado da execução, destacando os quatro testes efetuados com sucesso:



The screenshot shows a code editor with a file named `main.py`. The code defines three functions: `calcular_diferenca`, `calcular_energia`, and `validar_modo_viagem`. Below the code, there is a 'Run' button and a 'Share' button. The output area shows the result of running the tests: 'Ran 4 tests in 0.001s' and 'OK'.

```
main.py +
1 import unittest
2 from datetime import datetime
3
4 def calcular_diferenca(ano_atual, ano_destino):
5     # Retorna a diferença entre ano_destino e ano_atual.
6     # Exemplo: se ano_atual=2024 e ano_destino=2050, retorna 26.
7     # Essa função é essencial para diversos cálculos relacionados ao tempo, pois a partir da diferença é poss.
8     return ano_destino - ano_atual
9
10 def calcular_energia(anos):
11     # Recebe um número de anos e retorna a energia necessária, presumindo 10 unidades por ano.
12     # Exemplo: se anos=20, a função retorna 200.
13     # Esta lógica simples é um passo inicial para algo mais elaborado no futuro.
14     return abs(anos) * 10
15
16 def validar_modo_viagem(modo):
17     # Verifica se o modo de viagem informado é válido.
18     # Pode ser "instantaneo" ou "gradual". Caso contrário, retorna False.
19     # Essa verificação impede que o usuário selecione modos inexistentes, preservando a coerência do sistema.
20
Ln: 61, Col: 1
Run Share Command Line Arguments
-----
Ran 4 tests in 0.001s
OK
```

Figura 139 – Testes executados com sucesso

A adoção de `pytest` em vez de `unittest` traz uma abordagem mais simples e elegante para escrever e executar testes, uma vez que não é necessário criar classes de teste ou métodos especiais. O `pytest` detecta automaticamente funções cujo nome comece com `test_` e as executa, gerando relatórios claros e sucintos. Ele integra-se facilmente em fluxos de trabalho de desenvolvimento, sendo muitas vezes preferido por sua conveniência e extensibilidade.

No exemplo a seguir, as mesmas funções do exercício anterior (cálculo de diferença de anos, cálculo de energia e validação do modo de viagem) são testadas usando `pytest`. Em vez de uma classe, definimos funções normais de teste. Caso um teste falhe, o `pytest` exibirá uma mensagem clara, ajudando a encontrar o erro rapidamente. Para rodar esses testes, basta instalar o `pytest` (por exemplo, `pip install pytest`) e em seguida executar `pytest nome_do_arquivo.py` no terminal, sem precisar escrever nenhum código adicional para rodar o framework.


```
from datetime import datetime

def calcular_diferenca(ano_atual, ano_destino):
    # Calcula a diferença entre ano_destino e ano_atual, retornando um inteiro.
    # Exemplo: se ano_atual=2024 e ano_destino=2050, retorna 26.
    return ano_destino - ano_atual

def calcular_energia(anos):
    # Dado um número de anos, retorna a energia necessária, assumindo 10
    unidades por ano.
    # Exemplo: se anos=20, retorna 200.
    return abs(anos) * 10

def validar_modos_viagem(modos):
    # Verifica se o modo de viagem é "instantaneo" ou "gradual".
    # Retorna True para modos válidos, False caso contrário.
    modos = modos.strip().lower()
    return modos in ["instantaneo", "gradual"]

def test_calcular_diferenca():
    # Testa calcular_diferenca, esperando um resultado coerente.
    ano_atual = datetime.now().year
    ano_destino_teste = ano_atual + 30
    diferenca = calcular_diferenca(ano_atual, ano_destino_teste)
    assert diferenca == 30, f"Esperava-se 30, mas obteve-se {diferenca}."

def test_calcular_energia():
    # Testa calcular_energia, esperando que 30 anos resultem em 300 unidades de
    energia.
    diferenca = 30
    energia = calcular_energia(diferenca)
    assert energia == 300, f"Esperava-se 300, mas obteve-se {energia}."

def test_validar_modos_viagem_valido():
    # Testa validar_modos_viagem com um modo válido, "instantaneo", esperando
    True.
    resultado = validar_modos_viagem("instantaneo")
    assert resultado == True, "Esperava-se True para o modo 'instantaneo'."

def test_validar_modos_viagem_invalido():
    # Testa validar_modos_viagem com um modo inválido, "rapido", esperando False.
    resultado = validar_modos_viagem("rapido")
    assert resultado == False, "Esperava-se False para o modo 'rapido'."
```

Figura 140

No código anterior, cada função `test_*` realiza um teste simples e direto. O `pytest` detecta essas funções automaticamente. Ao executar `pytest` no terminal, o framework:

- procura funções que comecem com `test_`;
- executa cada teste, exibindo um ponto (.) para cada teste que passa;
- caso algum teste falhe, exibe detalhes da falha, incluindo o valor obtido e o esperado.

Se todos os testes passarem, o `pytest` exibirá algo como:

```
==== test session starts ====  
  
collected 4 items  
  
test_sistema_temporal.py .... [100%]  
  
==== 4 passed in 0.02s ====
```

Figura 141

Essa mensagem indica que todas as funções foram testadas com sucesso, aumentando a confiança na qualidade e coerência do código relacionado à manipulação temporal. Caso um teste falhe, o relatório do `pytest` ajudará a identificar o erro rapidamente.

Uma característica fundamental dos testes unitários é que eles devem ser automatizáveis e independentes. Automatizáveis significa que podem ser executados por ferramentas ou scripts sem intervenção humana, permitindo que o desenvolvedor execute centenas ou milhares de testes rapidamente. Independência implica que cada teste deve ser autossuficiente, não dependendo de outros testes ou de condições externas, como bancos de dados ou APIs, para funcionar corretamente. Essas características aumentam a confiabilidade e reduzem o risco de falhas causadas por fatores externos.

Além de verificar a funcionalidade do código existente, os testes unitários desempenham um papel importante no processo de **refatoração**. Quando o código é alterado para melhorar sua estrutura ou desempenho, os testes garantem que o comportamento do programa permaneça consistente. Eles também facilitam a introdução de novas funcionalidades, pois asseguram que as partes já testadas do programa continuam funcionando como esperado.

Em resumo, os testes unitários são uma prática essencial para garantir a qualidade e a confiabilidade do software. Eles são simples de implementar, altamente eficazes para detectar erros e fundamentais para manter a estabilidade do programa durante o desenvolvimento. Ao adotar testes automatizados, os desenvolvedores podem economizar tempo, reduzir custos e aumentar a confiança na robustez de seus sistemas. Python, com suas ferramentas integradas e bibliotecas dedicadas, torna a criação e execução de testes unitários acessível e eficiente para todos os níveis de programadores.

8 INTRODUÇÃO À PROGRAMAÇÃO ORIENTADA A OBJETOS (POO) EM PYTHON

A POO em Python é um paradigma que se apoia na ideia de modelar problemas e soluções por meio de entidades conhecidas como objetos, que por sua vez são instâncias de classes. Ao contrário de outros estilos de programação, como o estruturado ou funcional, a orientação a objetos busca organizar o código em torno desses objetos, que carregam consigo tanto dados quanto comportamentos. Esse modelo facilita a modularização, a manutenção e a expansão dos programas, tornando mais simples entender seu funcionamento à medida que as aplicações crescem em complexidade.

Em Python, o conceito de classe é o ponto de partida para trabalhar nesse paradigma. Uma classe pode ser vista como um molde que define características e ações comuns a um certo conjunto de elementos. Por exemplo, se desejamos representar funcionários de uma empresa, uma classe chamada "Funcionario" serviria para definir propriedades como nome, cargo e salário, bem como métodos que descrevam ações, tais como calcular bonificações ou exibir informações pessoais. Cada objeto criado a partir dessa classe é uma instância específica, com seus próprios valores de atributos, mantendo, no entanto, a mesma estrutura e conjunto de comportamentos definidos pela classe.

Um dos pilares da orientação a objetos é o encapsulamento, que consiste em agrupar dados e métodos relevantes dentro de uma mesma classe, protegendo o estado interno e tornando o código mais robusto.

Python também oferece mecanismos simples para realizar herança, permitindo a criação de novas classes que herdam atributos e métodos de classes já existentes. Assim, é possível adaptar ou estender funcionalidades sem ter de reescrever tudo do zero, promovendo a reutilização e a clareza do código.

Outro pilar da orientação a objetos é o polimorfismo, que possibilita a criação de métodos com o mesmo nome, porém comportamentos diferentes, dependendo do contexto ou do tipo do objeto. Essa propriedade incentiva a escrita de código genérico, capaz de lidar com diversas situações sem precisar de ramificações de lógica complexas. Além disso, Python adota uma filosofia flexível, o que significa que não há um sistema rígido de tipos e que o foco se volta mais para as capacidades (o que um objeto pode fazer) do que para sua classificação estática. Assim, quando um método é chamado em um objeto, o que importa é se esse objeto é capaz de responder àquela chamada, ou seja, se tem o comportamento exigido, em um estilo conhecido como duck typing.

A expressão duck typing vem de um ditado popular na língua inglesa: "if it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck" (se parece com um pato, nada como um pato e grasna como um pato, então provavelmente é um pato). Em termos de programação, isso significa que um objeto pode realizar a operação necessária, não importa qual seja sua classe ou tipo.

Por fim, a introdução à orientação a objetos em Python prepara o programador para criar aplicações mais bem estruturadas, fáceis de entender e manter, e escaláveis conforme as necessidades do projeto. Essa abordagem promove uma separação lógica de responsabilidades e incentiva uma melhor organização do código, ao mesmo tempo que oferece a flexibilidade e a clareza características do Python. O uso

de classes e objetos, encapsulamento, herança e polimorfismo é um dos pilares desse paradigma, e compreender esses conceitos é um passo fundamental para tirar proveito dos benefícios que a orientação a objetos pode trazer ao desenvolvimento de software.

8.1 Classes e objetos

Como o próprio nome destaca, na POO o objeto é um conceito extremamente relevante. **Objeto** é um ente, individualizado, do mundo real que pode ser utilizado na programação. Assim, uma mesa pode ser um objeto; uma caneta, um boleto bancário, um cartão de crédito, um passe escolar etc., todos podem ser considerados objetos.

O conceito de objeto na programação é mais abrangente que a noção de objetos que temos no mundo real, no qual associamos "coisas" como objetos e "seres" como outra categoria. No mundo real, não consideramos uma pessoa ou um gatinho como objetos, mas na POO eles podem ser. É perfeitamente possível ter uma pessoa ou gatinho modelado como objetos em um sistema computacional. Ainda é possível, sob a ótica computacional, que "pessoa" e "gato" sejam um único objeto, um exemplar (ou instância) de um "mamífero".

É possível observar no mundo real que um conjunto de objetos pode ter similaridade de características, coisas em comum, como se fizessem parte de um coletivo maior ou agrupamento. O objeto caneta que está em meu bolso é diferente do objeto caneta que está em seu estojo, que é distinto do objeto caneta que está na vitrine da loja. Esses três objetos têm formatos, tamanhos, marcas, diferentes, preço de aquisição, quantidade de tinta ou vida útil e talvez até cores de escrita diferentes. São entes individuais, portanto. Ainda assim, podemos chamar todos de caneta.

Em POO, podemos chamar caneta de **classe** e cada indivíduo representante da classe (cada uma das canetas) de **objeto** ou instância da classe caneta. Recapitulando o exemplo anterior, temos uma classe (caneta) e três objetos do tipo caneta. Então a classe é um conjunto de características comuns a todos os indivíduos (instâncias ou objetos).

Note ainda que, além das características citadas (tamanho e formato, marca, proprietário, valor de compra, quantidade de carga, vida útil, cor da tinta), existem ações ou comportamentos comuns: é esperado que uma caneta escreva (ação escrever) e eu posso transferir temporariamente a propriedade da caneta (ação emprestar), por exemplo.

Na linguagem Python, assim como em outras linguagens de POO, os atributos e métodos são componentes fundamentais para a definição e o funcionamento das classes e objetos. Os atributos (também conhecidos como campos) são as características ou propriedades dos objetos de uma classe, eles representam os dados que um objeto armazena.

Já os métodos são usados para definir comportamentos ou funcionalidades que os objetos de uma classe podem realizar. O fato de existir uma classe chamada caneta também garante que posso ter uma nova caneta se eu quiser. E pode ser diferente das anteriores, uma vez que é um indivíduo "caneta" ou instância "caneta". Do ponto de vista da programação, a classe existe desde o momento em que o

programador a digitou no código-fonte até o fim da vida do programa. Para ser mais preciso, a classe nasce um pouco antes: durante a modelagem, antes da programação.

As empresas de desenvolvimento de software usam a linguagem UML (Unified Modeling Language) para modelar as classes visualmente e depois inseri-las no código-fonte. Já o objeto tem um ciclo de vida diferente; ele nasce somente quando a programação do software foi finalizada e com o programa em execução.

Dentro do código, o programador inseriu uma espécie de semente que vai brotar quando o programa é executado. Falando de modo mais preciso, o objeto é criado na memória do computador assim que o programa em execução cria uma nova instância. E o objeto tem vida efêmera; ele será removido da memória quando terminar de cumprir seu papel.

Lembre-se que no computador a memória é volátil, ela está disponível para os programas em execução e perdura enquanto o equipamento estiver ligado. Veremos também que, no caso do Python, há um mecanismo de remoção automática de objetos em memória chamado GC (Garbage Collector).

Caso o programador não queira perder as informações do objeto, ele deverá utilizar estratégias de persistência (salvar em arquivos ou em bancos de dados, por exemplo). Retomando o exemplo da caneta: por se tratar de uma classe ela continuará existindo enquanto o programa existir. Já a caneta preta, marca Acme, do João, que custou R\$ 20,00, tem 6 meses de vida e metade da carga, será destruída ao término da execução do programa, caso não seja implementada a persistência no código-fonte.

Outro ponto relevante: só temos uma classe caneta, mas podemos criar diversas instâncias de canetas. O número máximo de objetos canetas que podemos criar vai depender do espaço em memória disponível para o programa e do tamanho em bits de cada caneta.

Em outras palavras, temos três diferenças fundamentais entre classes e objetos: o objeto é um representante da classe: pode-se criar múltiplos objetos a partir de uma única classe e cada um deles tem vida efêmera na memória volátil.



Saiba mais

A seguir destacamos dois livros amplamente reconhecidos como excelentes recursos para aprender e aprofundar o conhecimento em POO em Python.

A primeira indicação é uma referência de alta qualidade para programadores que desejam entender Python em profundidade, incluindo os princípios da POO. Ele aborda conceitos avançados de POO em Python, como herança, polimorfismo, metaclasses e protocolos. Além disso, explora como a POO se integra aos recursos mais dinâmicos de Python, como funções de primeira classe e decoradores:

RAMALHO, L. *Fluent Python: clear, concise, and effective programming*. Sebastopol: O'Reilly Media, 2022b.

O segundo é ideal para iniciantes e programadores intermediários que desejam aprender POO em Python de forma prática. Ele cobre desde os fundamentos, como classes e objetos, até tópicos avançados, como padrões de design e princípios de design orientado a objetos. O livro também aborda boas práticas e ferramentas para escrever código Python limpo e modular:

PHILLIPS, D. *Python 3 object-oriented programming*. Birmingham: Packt Publishing, 2010.

A exploração do continuum temporal não se limita a interações simples. A complexidade de uma aventura no tempo envolve múltiplos aspectos, desde a identidade do viajante e as coordenadas da linha temporal até os eventos históricos observados e a máquina utilizada para executar o salto. Em Python, a criação de classes e a instanciação de objetos permite modelar esses elementos de maneira estruturada, tornando o código mais claro, organizado e expansível.

A equipe que fez a modelagem do sistema para o viajante do tempo criou um diagrama de classes preliminar, em formato UML, para facilitar a visualização das informações levantadas:

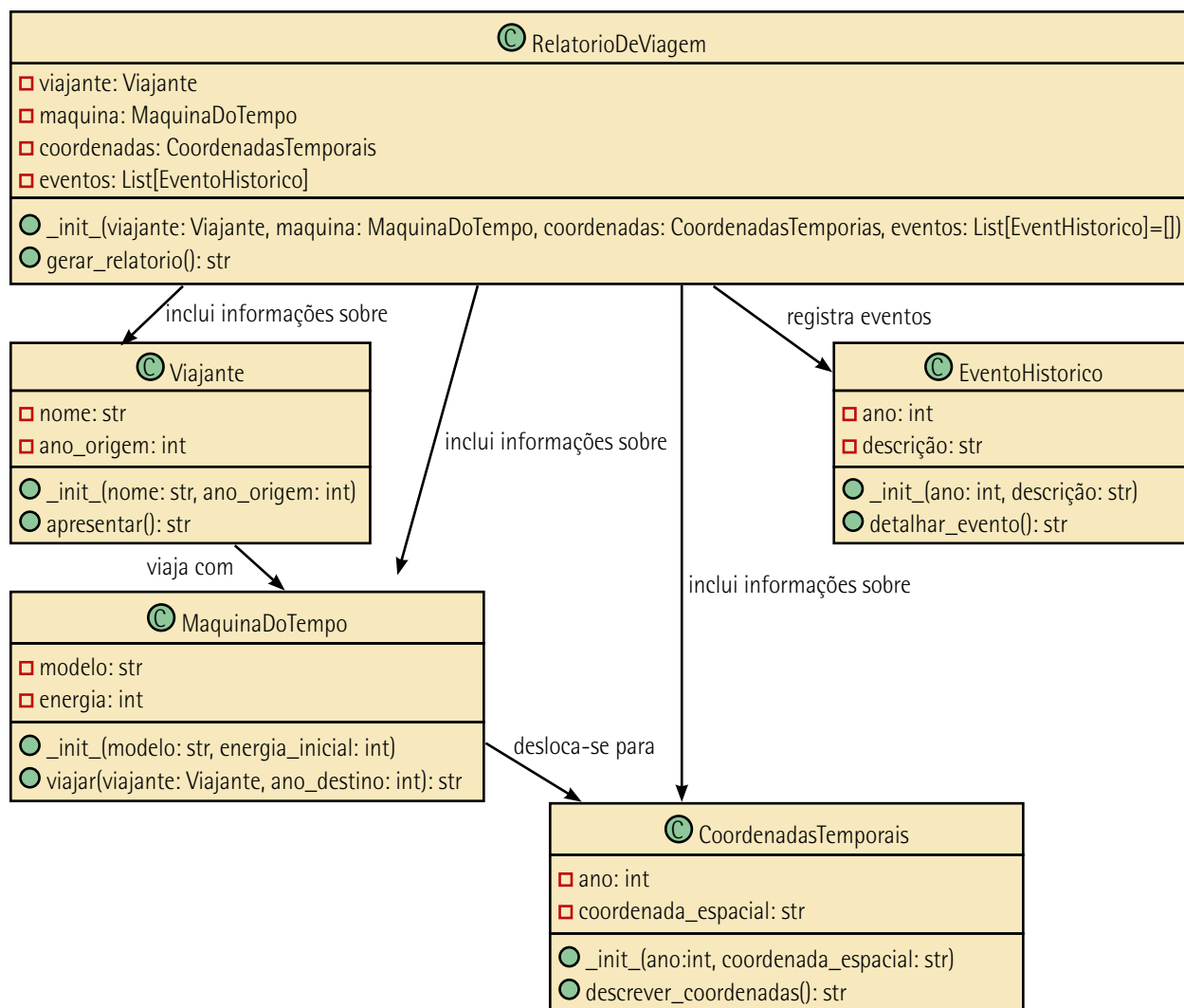


Figura 142 – Diagrama de classes preliminar desenhado pela equipe em formato UML

O código a seguir apresenta a implementação das cinco classes previstas no diagrama da figura anterior, cada uma representando um aspecto da simulação de viagens no tempo. Há uma classe para indicar o viajante, a máquina do tempo em si, as coordenadas temporais que descrevem o espaço-tempo a ser alcançado, um evento histórico para documentar o que é encontrado em cada época e um relatório de viagem que consolida as informações reunidas. Ao instanciar objetos dessas classes, é possível simular o preparo, a execução e a análise de uma jornada temporal.


```
from datetime import datetime

class Viajante:
    # Esta classe representa o indivíduo que viaja no tempo.
    # Ele possui um nome e o ano de origem do qual parte.
    # Poderia ter métodos para atualizar o ano atual do viajante conforme ele se desloca.
    def __init__(self, nome, ano_origem):
        self.nome = nome
        self.ano_origem = ano_origem

    def apresentar(self):
        # Apresenta quem é o viajante e de que ano ele vem.
        return f"Viajante {self.nome}, oriundo do ano {self.ano_origem}."

class MaquinaDoTempo:
    # Esta classe representa a máquina do tempo utilizada para deslocar-se entre as épocas.
    # Possui um método para viajar a um dado ano, alterando o estado do viajante.
    def __init__(self, modelo, energia_inicial):
        self.modelo = modelo
        self.energia = energia_inicial

    def viajar(self, viajante, ano_destino):
        # Deduz energia baseada na diferença entre anos.
        diferenca = abs(ano_destino - viajante.ano_origem)
        custo = diferenca * 10 # Exemplo simples: 10 unidades de energia por ano de diferença.
        if custo <= self.energia:
            self.energia -= custo
            viajante.ano_origem = ano_destino
            return f"A máquina do tempo {self.modelo} transportou o viajante para o ano {ano_destino}. Energia restante: {self.energia}"
        else:
            return "Energia insuficiente para realizar esta viagem temporal."

class CoordenadasTemporais:
    # Esta classe guarda informações sobre o ponto do continuum espaço-tempo a ser visitado.
    # Por simplicidade, armazenaremos apenas o ano e uma breve coordenada espacial fictícia.
    def __init__(self, ano, coordenada_espacial):
        self.ano = ano
        self.coordenada_espacial = coordenada_espacial

    def descrever_coordenadas(self):
        # Descreve o destino pretendido.
        return f"Coordenadas: Ano {self.ano}, Posição Espacial: {self.coordenada_espacial}"

class EventoHistorico:
    # Esta classe representa um evento encontrado em uma certa época.
    # Pode ser um fato histórico relevante, uma cerimônia, um descobrimento, etc.
    def __init__(self, ano, descricao):
        self.ano = ano
```

```

        self.descricao = descricao

    def detalhar_evento(self):
        # Retorna a descrição do evento, possibilitando o viajante entendê-lo.
        return f"No ano {self.ano}: {self.descricao}"

class RelatorioDeViagem:
    # Após uma série de deslocamentos, o viajante pode gerar um relatório.
    # Este relatório reúne informações sobre o viajante, a máquina, o destino e
    possíveis eventos observados.
    def __init__(self, viajante, maquina, coordenadas, eventos=[]):
        self.viajante = viajante
        self.maquina = maquina
        self.coordenadas = coordenadas
        self.eventos = eventos

    def gerar_relatorio(self):
        # Cria um resumo das informações da viagem.
        relatorio = []
        relatorio.append(self.viajante.apresentar())
        relatorio.append(f"Máquina do tempo utilizada: {self.maquina.modelo},
        Energia atual: {self.maquina.energia}")

        relatorio.append(self.coordenadas.descrever_coordenadas())
        if self.eventos:
            relatorio.append("Eventos observados durante a viagem:")
            for e in self.eventos:
                relatorio.append(" - " + e.detalhar_evento())
        else:
            relatorio.append("Nenhum evento notável foi registrado nesta época.")
        return "\n".join(relatorio)

# Código principal:
# Aqui criamos instâncias das classes definidas, simulando uma breve jornada.

# Cria um viajante chamado Alice, oriundo do ano atual.
ano_atual = datetime.now().year
viajante = Viajante("Alice", ano_atual)

# Cria a máquina do tempo "Chronos-2000" com energia inicial de 2000 unidades.
maquina = MaquinaDoTempo("Chronos-2000", 2000)

# Define coordenadas temporais: o viajante deseja ir para o ano 1800, em uma
região específica.
destino = CoordenadasTemporais(1800, "Setor Alfa-7")

# Antes de criar o relatório final, realizamos a viagem.
resultado_viagem = maquina.viajar(viajante, destino.ano)
print(resultado_viagem)

# Suponhamos que o viajante encontrou dois eventos marcantes no ano 1800.
evento1 = EventoHistorico(1800, "Assisti a uma feira de trocas de especiarias
exóticas.")
evento2 = EventoHistorico(1800, "Presenciei o início da construção de um
observatório astronômico.")

```

```
# Agora, cria-se um relatório da viagem.
relatorio = RelatorioDeViagem(viajante, maquina, destino, [evento1, evento2])
print(relatorio.gerar_relatorio())

# Ao rodar este código, veremos mensagens imprimindo o resultado da viagem e o
relatório.
# Embora seja um exemplo simples, temos aqui 5 classes: Viajante,
MaquinaDoTempo, CoordenadasTemporais, EventoHistorico e RelatorioDeViagem.
# Cada uma modela um aspecto da jornada temporal, e a instanciação dos objetos
mostra como orquestrar esses elementos.
# Com o passar do tempo, é possível adicionar métodos mais complexos, atributos
adicionais e lógicas sofisticadas,
# mantendo tudo bem organizado por meio da orientação a objetos.
```

Figura 143

Esse código modela uma jornada temporal usando conceitos fundamentais da POO, como classes, objetos, métodos e encapsulamento, para criar uma simulação organizada e modular. Cada classe representa uma entidade distinta da narrativa, enquanto os objetos são instâncias dessas classes que interagem para realizar a funcionalidade desejada. Vamos analisar em detalhes os aspectos da POO presentes no código.

A classe `Viajante` é a abstração de um indivíduo que realiza viagens no tempo. Ela tem atributos para armazenar o nome e o ano de origem do viajante, além de um método `apresentar`, que retorna uma descrição textual do viajante. Aqui, vemos o uso do método especial `__init__`, que é chamado automaticamente quando uma instância da classe é criada. Ele inicializa os atributos do objeto, garantindo que cada viajante tenha dados específicos e consistentes.

Para criar uma classe em Python, utiliza-se a palavra-chave `class`, seguida pelo nome da classe. O nome da classe, por convenção, é escrito em CamelCase, ou seja, com a primeira letra de cada palavra em maiúscula, como `Viajante`, `MaquinaDoTempo` ou `RelatorioDeViagem`. Dentro do corpo da classe, podem ser definidos atributos e métodos que descrevem o comportamento e as propriedades dos objetos que serão criados a partir dessa classe.

A criação de um objeto (ou instância de uma classe) é o processo de transformar a definição abstrata da classe em uma entidade concreta, com valores específicos para seus atributos. Isso é feito chamando a classe como se fosse uma função, passando os valores necessários para inicializar os atributos da instância por meio de seu método especial `__init__`.

Vamos usar a classe `Viajante` como exemplo para as demais:

- `class Viajante:` define a classe com o nome `Viajante`.
- O método especial `__init__` é usado para inicializar os atributos do objeto. Ele recebe `nome` e `ano_origem` como parâmetros e os atribui à instância por meio de `self.nome` e `self.ano_origem`. O `self` é uma referência ao próprio objeto sendo criado.

- O método `apresentar` é uma funcionalidade específica dessa classe. Ele utiliza os atributos da instância (`self.nome` e `self.ano_origem`) para retornar uma string descritiva.

Posteriormente, a classe `Viajante` é instanciada (ou seja, um objeto da classe é criado) com o seguinte código:

```
# Cria um viajante chamado Alice, oriundo do ano atual.  
ano_atual = datetime.now().year  
viajante = Viajante("Alice", ano_atual)
```

Figura 144

Nesse fragmento código:

- A variável `ano_atual` é inicializada como o ano atual, obtido usando `datetime.now().year`.
- O objeto `viajante` é criado ao chamar a classe `Viajante` com os argumentos "Alice" e `ano_atual`.
- Durante essa chamada, o método `__init__` da classe `Viajante` é executado automaticamente, atribuindo "Alice" a `self.nome` e o valor de `ano_atual` a `self.ano_origem`.

A classe `MaquinaDoTempo` modela a máquina utilizada para viajar entre épocas. Ela tem atributos para armazenar o modelo da máquina e sua energia restante, além de um método `viajar`, que realiza o deslocamento temporal. O método `viajar` encapsula a lógica necessária para verificar se há energia suficiente para a viagem, calcular o custo com base na diferença de anos e ajustar os atributos do viajante e da máquina. O encapsulamento dessa lógica dentro do método `viajar` é um exemplo de boa prática, pois mantém o código modular e reutilizável.

A classe `CoordenadasTemporais` representa o destino da viagem, incluindo o ano e uma localização espacial fictícia. Ela tem um método `descrever_coordenadas`, que retorna uma descrição formatada do destino. Esse tipo de classe ajuda a organizar as informações relacionadas ao contexto temporal e espacial de maneira clara e específica.

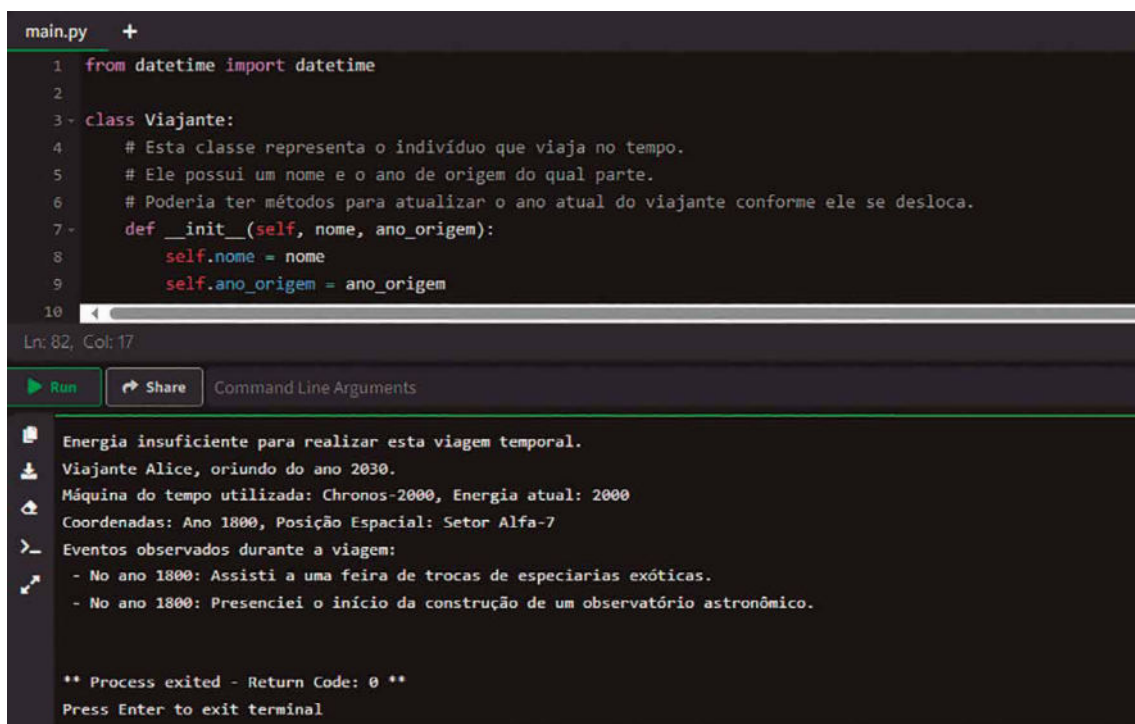
A classe `EventoHistorico` abstrai eventos significativos observados durante a viagem no tempo. Ela tem atributos para o ano e a descrição do evento, além de um método `detalhar_evento`, que retorna uma descrição legível do evento. Essa classe encapsula os detalhes dos eventos, permitindo que eles sejam facilmente registrados e exibidos no contexto de uma viagem.

A classe `RelatorioDeViagem` reúne todas as informações relevantes de uma viagem temporal, como o viajante, a máquina, o destino e os eventos observados. Ela demonstra a capacidade da POO de organizar dados e comportamentos de maneira hierárquica. O método `gerar_relatorio` compila essas informações em um formato de texto legível, integrando as descrições fornecidas pelos métodos de outras classes, como `apresentar`, `descrever_coordenadas` e `detalhar_evento`. Isso destaca a interação entre objetos e o conceito de **composição**, no qual **uma classe usa objetos de outras classes para compor sua funcionalidade**.

No código principal, vemos a criação de **instâncias dessas classes** e sua interação para realizar a simulação. O objeto viajante é criado com o nome "Alice" e o ano atual, enquanto o objeto máquina é uma instância da classe `MaquinaDoTempo` com um modelo e energia inicial. As coordenadas temporais são representadas por uma instância de `CoordenadasTemporais`, e eventos históricos são registrados como instâncias da classe `EventoHistorico`.

O método `viajar` da máquina do tempo é chamado para realizar a viagem ao destino especificado. Após a viagem, um relatório é gerado pela classe `RelatorioDeViagem`, mostrando como os dados das diferentes entidades são integrados e apresentados de maneira organizada.

A figura a seguir ilustra a execução desse programa:



```
main.py +
1 from datetime import datetime
2
3 class Viajante:
4     # Esta classe representa o indivíduo que viaja no tempo.
5     # Ele possui um nome e o ano de origem do qual parte.
6     # Poderia ter métodos para atualizar o ano atual do viajante conforme ele se desloca.
7     def __init__(self, nome, ano_origem):
8         self.nome = nome
9         self.ano_origem = ano_origem
10
Ln: 82, Col: 17
Run Share Command Line Arguments
Energia insuficiente para realizar esta viagem temporal.
Viajante Alice, oriundo do ano 2030.
Máquina do tempo utilizada: Chronos-2000, Energia atual: 2000
Coordenadas: Ano 1800, Posição Espacial: Setor Alfa-7
Eventos observados durante a viagem:
- No ano 1800: Assisti a uma feira de trocas de especiarias exóticas.
- No ano 1800: Presenciei o início da construção de um observatório astronômico.
** Process exited - Return Code: 0 **
Press Enter to exit terminal
```

Figura 145 – Execução do código que implementa as cinco classes

Esse código exemplifica como a POO ajuda a organizar e modularizar sistemas complexos. Cada classe encapsula responsabilidades específicas, e os objetos interagem de maneira lógica, promovendo legibilidade, reutilização e expansão. Tal design permite que novos recursos sejam adicionados facilmente, como novos tipos de máquinas, modos de viagem ou análises mais detalhadas de eventos históricos, sem comprometer a estrutura existente.

8.2 Métodos e encapsulamento

O encapsulamento refere-se à capacidade de ocultar os detalhes internos de um objeto e expor apenas uma interface pública. Isso significa que os atributos (dados) de um objeto são geralmente definidos como privados e o acesso a esses atributos é controlado por métodos públicos.

Essa prática ajuda a manter a integridade dos dados e facilita a manutenção do código, pois as mudanças internas podem ser feitas sem afetar o código que utiliza o objeto. Tal princípio permite que o programador defina os atributos de uma classe como privados, o que significa que esses dados não podem ser acessados diretamente de fora da classe. Isso é essencial para evitar que dados sensíveis ou críticos sejam manipulados incorretamente ou indevidamente. Por exemplo, imagine uma classe `ContaBancaria` que tem um atributo de saldo. Ao tornar o saldo privado, há um impedimento para que qualquer código externo modifique o saldo diretamente, o que poderia levar a resultados indesejados.

Para permitir que os objetos dessa classe sejam usados, podemos definir métodos públicos que atuam como uma interface para interações externas. Esses métodos são responsáveis por acessar ou modificar os atributos privados de forma controlada e segura.

No caso da classe `ContaBancaria`, podemos criar métodos públicos como `depositar` e `sacar` que permitem que os usuários da classe interajam com o saldo de maneira controlada e segura. Esses métodos são chamados de interface pública. O encapsulamento oferece controle sobre como os dados são acessados e modificados. Isso significa que podemos impor regras e validações para garantir que os dados permaneçam em um estado consistente. No exemplo da `ContaBancaria`, o método `sacar` pode verificar se o saldo é suficiente antes de permitir uma retirada. Se o saldo for insuficiente, ele pode gerar um erro ou uma exceção, garantindo que o saldo nunca fique negativo.

Outra vantagem do encapsulamento é que ele facilita a manutenção do código. Como os detalhes internos de uma classe são encapsulados e não expostos diretamente, podemos fazer alterações internas, como a estrutura de dados usada para armazenar os atributos, sem afetar o código externo que usa a classe. Isso é conhecido como "separação de preocupações" e ajuda a isolar as mudanças internas para evitar efeitos colaterais não desejados.

O encapsulamento também permite a evolução e o aprimoramento das classes com mais facilidade. Se for necessário adicionar novos recursos ou validar dados de maneira diferente, podemos fazer isso dentro dos métodos públicos sem afetar o código que usa a classe.

Ademais, o encapsulamento promove a reutilização de código. É possível criar uma classe bem encapsulada e, em seguida, usá-la em diferentes partes do programa sem se preocupar com os detalhes internos. Isso economiza tempo e esforço, pois não será preciso reescrever o mesmo código repetidamente. Ao ocultar os detalhes internos de um objeto, o encapsulamento ajuda a proteger o estado do objeto contra alterações não autorizadas. Isso é importante para garantir a integridade dos dados e a segurança do programa.

Em Python, a criação de métodos e o controle de acesso aos dados são conceitos fundamentais da POO, permitindo modelar entidades complexas como viajantes no tempo, máquinas do tempo e coordenadas temporais. Métodos encapsulam comportamentos e ações que os objetos podem realizar, enquanto o controle de acesso garante que certos atributos internos estejam protegidos contra alterações indevidas, preservando a consistência da lógica da viagem no tempo.

Métodos são funções definidas dentro de uma classe que operam sobre os atributos do objeto. Eles permitem que objetos como viajantes ou máquinas do tempo interajam com o mundo fictício em que estão inseridos. O primeiro parâmetro de qualquer método é `self`, que representa a instância atual do objeto, permitindo acessar seus atributos e outros métodos.



Lembrete

Python fornece métodos padrão para seus tipos embutidos:

- **Strings** (str): `upper()`, `lower()`, `strip()`, `replace()`, `split()`.
- **Listas** (list): `append()`, `extend()`, `pop()`, `sort()`.
- **Dicionários** (dict): `get()`, `keys()`, `values()`, `items()`.

Nos exemplos deste livro-texto já utilizamos vários deles.

Um exemplo clássico de método é o **método de instância**, que opera sobre um viajante específico. Por exemplo, o método `apresentar` de um viajante pode retornar uma descrição do explorador temporal e seu ano de origem:

```
class Viajante:
    def __init__(self, nome, ano_origem):
        self.nome = nome
        self.ano_origem = ano_origem

    def apresentar(self):
        return f"Viajante {self.nome}, oriundo do ano {self.ano_origem}."
```

Figura 146

Nesse caso, `apresentar` usa os atributos `nome` e `ano_origem` do viajante para compor uma mensagem descritiva.

Além disso, há **métodos de classe** que operam no contexto da classe como um todo, em vez de uma instância específica. Por exemplo, a classe `MaquinaDoTempo` poderia incluir um método que retorna um modelo padrão:

```
class MaquinaDoTempo:
    @classmethod
    def modelo_padrao(cls):
        return cls("Chronos-1000", energia_inicial=5000)
```

Figura 147

Métodos estáticos, por sua vez, não dependem de atributos da instância ou da classe. Um exemplo seria calcular o custo energético de uma viagem temporal com base na diferença de anos:

```
class MaquinaDoTempo:
    @staticmethod
    def calcular_custo(diferenca_anos):
        return diferenca_anos * 10
```

Figura 148

Esses métodos ajudam a encapsular comportamentos relevantes, garantindo que a lógica da viagem temporal seja fácil de entender e manter.

O controle de acesso aos dados é essencial para proteger a integridade das entidades temporais. Python permite implementar diferentes níveis de acesso por meio de convenções e modificadores de visibilidade, garantindo que apenas os métodos apropriados possam alterar ou acessar atributos específicos.

- **Atributos públicos:** por padrão, todos os atributos de um objeto são públicos. Por exemplo, o modelo de uma máquina do tempo ou o ano de origem de um viajante pode ser acessado diretamente:

```
maquina = MaquinaDoTempo("Chronos-2000", energia_inicial=3000)
print(maquina.modelo) # Acesso direto ao atributo público
```

Figura 149

- **Atributos protegidos:** para atributos que devem ser usados apenas dentro da classe ou suas subclasses, utiliza-se um único sublinhado `_` no início do nome. Por exemplo, a integridade estrutural de uma máquina do tempo pode ser considerada sensível:

```
class MaquinaDoTempo:
    def __init__(self, modelo):
        self._integridade_estrutural = 100
```

Figura 150

- **Atributos privados:** para proteger completamente um atributo, utiliza-se um duplo sublinhado `__`. Esse mecanismo de name mangling torna o atributo acessível apenas dentro da classe. Por exemplo:

```
class CoordenadasTemporais:
    def __init__(self, ano, posicao):
        self.__ano = ano
```

Figura 151

Para permitir acesso controlado, métodos públicos, conhecidos como getters e setters, podem ser criados:

```
class CoordenadasTemporais:
    def __init__(self, ano):
        self.__ano = ano

    def get_ano(self):
        return self.__ano

    def set_ano(self, novo_ano):
        if novo_ano > 0:
            self.__ano = novo_ano
        else:
            raise ValueError("O ano deve ser maior que zero.")
```

Figura 152

Esse controle é fundamental para evitar que dados críticos, como posição ou ano de uma coordenada temporal, sejam alterados de maneira indevida, comprometendo a integridade da viagem.

O encapsulamento, um princípio essencial da POO, permite que as classes ocultem detalhes internos e exponham apenas o que é necessário. Por exemplo, o método viajar de uma máquina do tempo poderia encapsular toda a lógica de cálculo de energia e alteração de ano, sem expor diretamente esses detalhes para o programador ou usuário:

```
class MaquinaDoTempo:
    def __init__(self, modelo, energia_inicial):
        self.modelo = modelo
        self.__energia = energia_inicial

    def viajar(self, viajante, ano_destino):
        diferenca = abs(ano_destino - viajante.ano_origem)
        custo = diferenca * 10
        if custo <= self.__energia:
            self.__energia -= custo
            viajante.ano_origem = ano_destino
            return f"Viajante transportado para o ano {ano_destino}. Energia restante: {self.__energia}."
        else:
            return "Energia insuficiente."
```

Figura 153

Em resumo, a criação de métodos e o controle de acesso em Python são ferramentas poderosas para modelar sistemas complexos, como viagens no tempo. Métodos encapsulam comportamentos específicos das entidades, enquanto o controle de acesso protege os dados contra manipulações indevidas, garantindo consistência e robustez no design do programa.



Resumo

Esta unidade abordou os processos de depuração e teste no desenvolvimento de software com Python, destacando sua importância para garantir o funcionamento correto e eficiente de algoritmos. A depuração foi apresentada como o processo de localizar e corrigir erros no código, utilizando técnicas que variam desde o uso de mensagens simples com a função `print` até ferramentas avançadas como o depurador integrado `pdb`. A depuração visual, oferecida por IDEs como PyCharm e Visual Studio Code, também é mencionada como um recurso que facilita a identificação de problemas em projetos complexos.

Também enfatizamos a importância dos testes automatizados, com foco nos testes unitários. Eles verificam se pequenas partes do código, como funções ou métodos, funcionam corretamente em isolamento. Python oferece bibliotecas integradas como `unittest` e frameworks mais simples, como `pytest`, que permitem escrever e executar testes de forma eficiente. Exemplos incluem a validação de funções que calculam diferenças de anos, energia necessária para viagens temporais e modos de viagem válidos.

Em seguida, exploramos o paradigma da POO em Python, acentuando seus fundamentos, vantagens e aplicações práticas. Esse paradigma organiza o código em torno de objetos, que são instâncias de classes, combinando dados e comportamentos em entidades estruturadas.

Introduzimos as classes como moldes que definem atributos e métodos comuns a objetos de um mesmo tipo. Utilizando exemplos didáticos, como uma classe para modelar "canetas" e suas instâncias, demonstramos como objetos individuais podem ter características próprias (atributos) e ações específicas (métodos). A relação entre classes e objetos foi esclarecida, evidenciando que a classe é uma definição genérica, enquanto o objeto é uma ocorrência concreta que reside na memória durante a execução do programa.

Entre os pilares da POO, destacam-se o encapsulamento, que é descrito como a prática de proteger o estado interno de objetos, expondo apenas interfaces públicas para interação externa. O encapsulamento é aprofundado com exemplos que mostram como proteger atributos por meio de convenções e modificadores de visibilidade, como o uso de sublinhados em nomes de atributos para indicar proteção ou privacidade.



Exercícios

Questão 1. Considere um aplicativo para e-commerce em Python, chamado `app.py`, que calcula o preço total de um pedido aplicando descontos específicos. Segue seu código:

```
# arquivo: app.py
def calcular_preco(pedido):
    print("DEBUG: Iniciando cálculo de preço...")
    valor_bruto = sum(item["quantidade"] * item["preco"] for item in
pedido["itens"])
    print(f"DEBUG: Valor bruto calculado: {valor_bruto}")
    desconto = 0
    if pedido.get("cupom") == "PROMO10":
        desconto = valor_bruto * 0.1
        print(f"DEBUG: Desconto aplicado: {desconto}")
    valor_final = valor_bruto - desconto
    print(f"DEBUG: Valor final: {valor_final}")
    return valor_final
```

Figura 154

O desenvolvedor adicionou testes automatizados usando `unittest` e, para fins de depuração, incluiu alguns `print` internos no código do cálculo de `app.py`. Como ele estava usando um interpretador Python online, sem acesso direto ao comando `python -m unittest`, o teste foi executado programaticamente ao final do arquivo de teste, chamando o `unittest.main()` diretamente:

```
# arquivo: teste_preco.py
import unittest
from app import calcular_preco

class TesteCalculoPreco(unittest.TestCase):
    def test_calculo_com_cupom(self):
        pedido = {
            "itens": [
                {"quantidade": 2, "preco": 100},
                {"quantidade": 1, "preco": 200}
            ],
            "cupom": "PROMO10"
        }
        resultado = calcular_preco(pedido)
        self.assertEqual(resultado, 360.0)

if __name__ == "__main__":
    unittest.main()
```

Figura 155

Analise os códigos `app.py` e `teste_preco.py`. Imagine que o teste seja executado clicando no botão Run do interpretador online e assinale a alternativa que corresponde à saída completa exibida no terminal ao término do teste. Considere que não há nenhum outro teste definido além do apresentado e que não há falhas de sintaxe.

A).

```
-----  
Ran 1 test in 0.000s  
OK  
DEBUG: Iniciando cálculo de preço...  
DEBUG: Valor bruto calculado: 400  
DEBUG: Desconto aplicado: 40.0  
DEBUG: Valor final: 360.0
```

B) F

```
=====  
FAIL: test_calculo_com_cupom (__main__.TesteCalculoPreco)  
-----  
Traceback (most recent call last):  
  File "teste_preco.py", line 14, in test_calculo_com_cupom  
    self.assertEqual(resultado, 360.0)  
AssertionError: 400.0 != 360.0  
-----  
Ran 1 test in 0.000s  
FAILED (failures=1)  
DEBUG: Iniciando cálculo de preço...  
DEBUG: Valor bruto calculado: 400  
DEBUG: Desconto aplicado: 40.0  
DEBUG: Valor final: 360.0
```

C).

```
-----  
Ran 1 test in 0.000s  
OK
```

D).

```
-----  
Ran 1 test in 0.000s  
OK  
DEBUG: Iniciando cálculo de preço...  
DEBUG: Valor bruto calculado: 400  
DEBUG: Valor final: 400
```

E) E

```
=====
ERROR: test_calculo_com_cupom (__main__.TesteCalculoPreco)
-----
Traceback (most recent call last):
  File "teste_preco.py", line 13, in test_calculo_com_cupom
    resultado = calcular_preco(pedido)
NameError: name 'calcular_preco' is not defined
-----
Ran 1 test in 0.000s
FAILED (errors=1)
DEBUG: Iniciando cálculo de preço...
DEBUG: Valor bruto calculado: 400
DEBUG: Desconto aplicado: 40.0
DEBUG: Valor final: 360.0
```

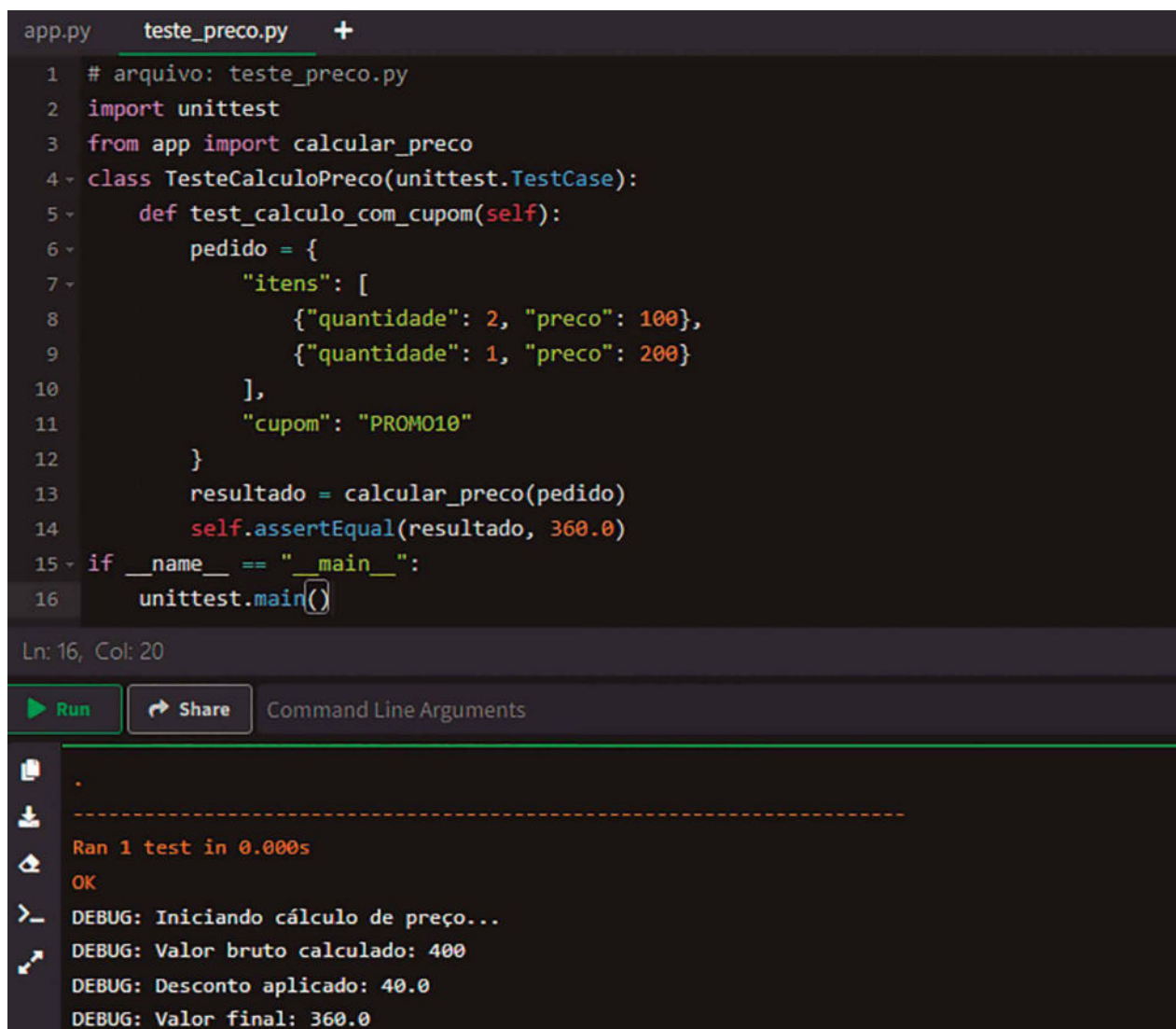
Resposta correta: alternativa A.

Análise da questão

A análise da saída envolve entender o fluxo do código. Primeiro, o unittest instancia a classe TesteCalculoPreco e executa o método test_calculo_com_cupom. Esse teste cria um dicionário pedido com itens e um cupom PROMO10. Ao chamar calcular_preco, serão impressas as mensagens de depuração: o valor bruto ($2 \times 100 + 1 \times 200 = 400$), o desconto de 10% sobre 400 (40.0) e o valor final (360.0).

Como o teste verifica self.assertEqual(resultado, 360.0) e esse é exatamente o valor retornado por calcular_preco, o teste passa com sucesso. Em seguida, o unittest imprime um ponto (.), o que indica que o teste teve êxito, seguido do resumo Ran 1 test in 0.000s e OK. A única alternativa que apresenta esse exato fluxo de saída (o ponto, depois o resumo e OK e, por fim, as mensagens de depuração) é a alternativa A.

Atendendo ao cenário proposto na questão, utilizamos um interpretador online (disponível em <https://www.online-python.com/>). conforme apresentado na imagem a seguir. Na parte superior da imagem, vemos o arquivo app.py e o arquivo teste_preco.py (respectivamente, com seus códigos). Na sequência, pressionamos o botão "Run". Os resultados aparecem na parte inferior da imagem:



```

app.py  teste_preco.py  +
1  # arquivo: teste_preco.py
2  import unittest
3  from app import calcular_preco
4  class TesteCalculoPreco(unittest.TestCase):
5      def test_calculo_com_cupom(self):
6          pedido = {
7              "itens": [
8                  {"quantidade": 2, "preco": 100},
9                  {"quantidade": 1, "preco": 200}
10             ],
11             "cupom": "PROM010"
12         }
13         resultado = calcular_preco(pedido)
14         self.assertEqual(resultado, 360.0)
15  if __name__ == "__main__":
16      unittest.main()

```

Ln: 16, Col: 20

Run Share Command Line Arguments

```

.
-----
Ran 1 test in 0.000s
OK
DEBUG: Iniciando cálculo de preço...
DEBUG: Valor bruto calculado: 400
DEBUG: Desconto aplicado: 40.0
DEBUG: Valor final: 360.0

```

Figura 156

Análise das alternativas

A) Alternativa correta.

Justificativa: exceto a alternativa A, todas introduzem cenários de falha, ausência de mensagens ou erros de lógica que não condizem com o fluxo real do código. As demais alternativas apresentadas diferem em aspectos cruciais que tornam suas saídas incompatíveis com o comportamento correto do código.

B) Alternativa incorreta.

Justificativa: sugere que o teste falhou, exibindo um F ao invés de um ponto. Além disso, indica um erro de asserção, sugerindo que o valor esperado era diferente do obtido, o que não corresponde à lógica

do código analisado: o desconto e o valor final foram calculados corretamente, tornando improvável um `AssertionError`.

C) Alternativa incorreta.

Justificativa: ignora completamente as mensagens de depuração impressas pela função `calcular_preco`, apresentando apenas a saída padrão do `unittest` (um ponto e o resumo dos testes). Como o código explicitamente contém `print` statements dentro da função, essas linhas deveriam aparecer com o resultado do teste, tornando a ausência delas irrealista.

D) Alternativa incorreta.

Justificativa: mostra a execução do teste sem aplicar o desconto, sugerindo que o valor final é igual ao valor bruto (400.0). Essa saída ignora o cupom `PROMO10` e a lógica de desconto, que estão claramente implementadas no código.

E) Alternativa incorreta.

Justificativa: apresenta um erro de execução (`NameError`) ao afirmar que `calcular_preco` não foi definido. Isso não faz sentido, já que o código fornece a definição de `calcular_preco` no arquivo `app.py` e a importa corretamente em `teste_preco.py`.

Questão 2. Considere o código Python a seguir, que faz parte de um aplicativo para gerenciamento de funcionários da empresa `alunosACME`. O código define uma classe `Funcionario` com atributos privados, métodos, modificadores e uma lógica interna de cálculo de bônus. Além disso, há interações entre objetos e alterações dos atributos que testam o conceito de encapsulamento.

```
class Funcionario:
    def __init__(self, nome, salario):
        self.__nome = nome
        self.__salario = salario
        self.__historico_acoes = []

    def registrar_acao(self, descricao):
        self.__historico_acoes.append(descricao)

    def get_nome(self):
        return self.__nome

    def set_salario(self, valor):
        if valor > 0:
            self.__salario = valor
        else:
            # Ignora valores não positivos
            pass

    def calcular_bonus(self):
        # Bônus baseado no número de ações registradas
        if len(self.__historico_acoes) >= 3:
            return self.__salario * 0.1
        return 0
```

```
def resumo(self):
    return (f"Funcionario: {self.__nome}, "
            f"Salário: {self.__salário}, "
            f"Bônus: {self.calcular_bonus()}, "
            f"Ações: {self.__historico_acoes}")

# Código principal:
f1 = Funcionario("Alice", 5000)
f1.registrar_acao("Finalizar relatório")
f1.registrar_acao("Atender cliente")
f1.set_salario(5500) # Altera o salário, pois é positivo
f1.registrar_acao("Apresentar proposta")
f1.registrar_acao("Negociar contrato")

f2 = Funcionario("Bob", 4000)
f2.registrar_acao("Receber treinamento")
f2.set_salario(-100) # Valor negativo, não altera o salário
f2.registrar_acao("Assistir webinar")

print(f1.resumo())
print(f2.resumo())
```

Figura 157

Analise o código e assinale a alternativa que representa corretamente a saída final impressa no console após a execução.

- A) **Funcionario: Alice**, Salário: 5500, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento', 'Assistir webinar']
- B) **Funcionario: Alice**, Salário: 5000, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento', 'Assistir webinar']
- C) **Funcionario: Alice**, Salário: 5500, Bônus: 0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento']
- D) **Funcionario: Alice**, Salário: 5500, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 400.0, Ações: ['Receber treinamento', 'Assistir webinar']
- E) **Funcionario: Alice**, Salário: 5000, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento', 'Assistir webinar']

Resposta correta: alternativa A.

Análise da questão

Para compreender a saída final, é necessário analisar o que acontece com cada objeto `Funcionario`. No caso de `f1` (Alice), ela inicia com salário 5000 e registra duas ações: `Finalizar relatório` e `Atender cliente`. Em seguida, o método `set_salario(5500)` é chamado com um valor positivo, o que atualiza o salário de Alice para 5500. Depois disso, são registradas mais duas ações: `Apresentar proposta` e `Negociar contrato`. Ao final, Alice tem o total de quatro ações. Segundo a lógica de bônus, se o funcionário tiver 3 ou mais ações, ganhará 10% do salário como bônus. Com salário 5500, o bônus é 550.0. Portanto, o resumo de Alice é:

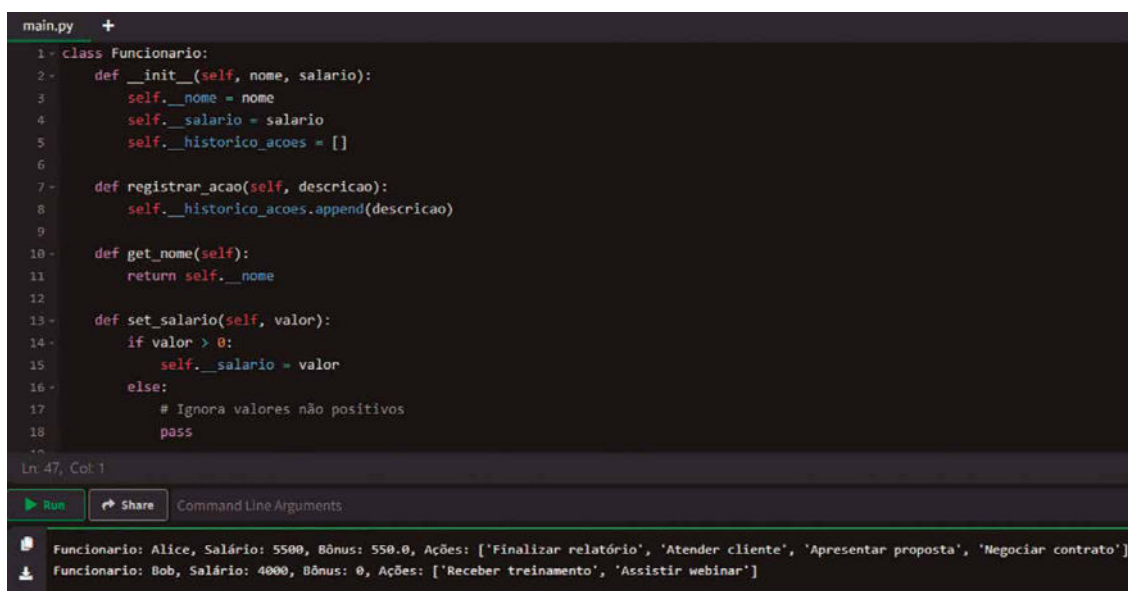
```
Funcionario: Alice, Salário: 5500, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
```

Para `f2` (Bob), ele começa com salário 4000 e registra uma ação: `Receber treinamento`. Em seguida, o código tenta alterar o salário para -100, mas esse valor é ignorado, mantendo o salário em 4000. Depois, é registrada mais uma ação: `Assistir webinar`. Agora, Bob tem duas ações. Com apenas duas ações, o bônus é 0. Assim, o resumo de Bob é:

```
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento', 'Assistir webinar']
```

Apenas a alternativa A descreve exatamente o estado final dos objetos, incluindo a atualização do salário de Alice para 5500, o bônus de 10% (550.0) e todas as quatro ações listadas. Além disso, apresenta corretamente as duas ações de Bob, seu salário inalterado (4000) e bônus zero. Portanto, a resposta correta é a alternativa A.

Na figura a seguir, vemos o código e o resultado da execução do programa:



```
main.py +
1 class Funcionario:
2     def __init__(self, nome, salario):
3         self.__nome = nome
4         self.__salario = salario
5         self.__historicoacoes = []
6
7     def registraracao(self, descricao):
8         self.__historicoacoes.append(descricao)
9
10    def get_nome(self):
11        return self.__nome
12
13    def set_salario(self, valor):
14        if valor > 0:
15            self.__salario = valor
16        else:
17            # Ignora valores não positivos
18            pass
19
20
Ln 47, Col 1
Run Share Command Line Arguments
Funcionario: Alice, Salário: 5500, Bônus: 550.0, Ações: ['Finalizar relatório', 'Atender cliente', 'Apresentar proposta', 'Negociar contrato']
Funcionario: Bob, Salário: 4000, Bônus: 0, Ações: ['Receber treinamento', 'Assistir webinar']
```

Figura 158

REFERÊNCIAS

- ALVES, E. *Estruturas de dados com Python*. São Paulo: Novatec, 2022.
- ARAÚJO, G. *Automatize tarefas maçantes com Python*. São Paulo: Novatec, 2020.
- BOOCH, G. *Object-oriented analysis and design with applications*. 2. ed. Redwood City: Benjamin/Cummings, 1994.
- CORMEN, H. et al. *Algoritmos: teoria e prática*. 4. ed. Rio de Janeiro: LTC, 2024.
- FURTADO, A. L. *Python: programação para leigos*. Rio de Janeiro: Alta Books, 2021.
- GÉRON, A. *Mãos à obra: aprendizado de máquina com Scikit-Learn, Keras & TensorFlow – conceitos, ferramentas e técnicas para a construção de sistemas inteligentes*. 2. ed. São Paulo: Alta Books, 2021.
- GRUS, J. *Data Science do zero: noções fundamentais com Python*. 2. ed. São Paulo: Alta Books, 2021.
- KNUTH, D. *The art of computer programming*. 3. ed. Londres: Addison-Wesley Professional, 1997.
- MCKINNEY, W. *Python para análise de dados: tratamento de dados com Pandas, NumPy & Jupyter*. 3. ed. São Paulo: Novatec, 2023.
- NILO, L. E. *Introdução à programação com Python*. São Paulo: Novatec, 2019.
- PATTERSON, D.; HENNESSY, J. *Computer organization and design RISC-V edition: the hardware software interface*. Burlington: Morgan Kaufmann Publishers, 2020.
- PHILLIPS, D. *Python 3 object-oriented programming*. Birmingham: Packt Publishing, 2010.
- PLATÃO. Laques. In: PLATÃO. *Diálogos*. Tradução: Edson Bini. São Paulo: Edipro, 2007.
- RAMALHO, L. *Fluent Python*. 2. ed. Sebastopol: O'Reilly Media, 2022a.
- RAMALHO, L. *Fluent Python: clear, concise, and effective programming*. Sebastopol: O'Reilly Media, 2022b.
- RAMALHO, L. *Python fluente: programação clara, concisa e eficaz*. São Paulo: Novatec, 2015.
- SILVEIRA, B. R. *Desenvolvimento web com Python e Flask*. São Paulo: Casa do Código, 2021.
- SWEIGART, A. *Automatize tarefas maçantes com Python: programação prática para verdadeiros iniciantes*. São Paulo: Novatec, 2015.
- VAN ROSSUM, G.; DRAKE, F. L. *Python Reference Manual*. PythonLabs, 2007.

VIEIRA, A. *Introdução à ciência da computação com Python*. Rio de Janeiro: LTC, 2020.

WING, J. M. Computational thinking. *Communications of the ACM*, v. 49, n. 3, p. 33-35, 2006.

ZAVAGLIA, F. *Lógica de programação: a construção de algoritmos e estruturas de dados*. São Paulo: Érica, 2017.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no vertical margin lines or other markings present. The paper appears to be a standard piece of stationery used for writing or drawing.